

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

GRADUATION THESIS

ShieldX: An Intelligent Endpoint Security Agent for Malware Detection over DNS-over-HTTPS using Machine Learning

NGUYEN QUANG TRUNG
trung.nq225557@sis.hust.edu.vn

Program: Cyber Security

Supervisor: PhD. Nguyen Huu Duc

Department: Computer Engineering

School: School of Information and Communications Technology

HANOI, 06/2026

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

GRADUATION THESIS

ShieldX: An Intelligent Endpoint Security Agent for Malware Detection over DNS-over-HTTPS using Machine Learning

NGUYEN QUANG TRUNG
trung.nq225557@sis.hust.edu.vn

Program: Cyber Security

Supervisor: PhD. Nguyen Huu Duc _____

Signature

Department: Computer Engineering

School: School of Information and Communications Technology

HANOI, 06/2026

ACKNOWLEDGMENTS

First and foremost, I would like to express my deepest gratitude to my supervisor, PhD. Nguyen Huu Duc, for his invaluable guidance, continuous support, and insightful feedback throughout the course of this graduation thesis. His expertise in network security and machine learning has been instrumental in shaping this work.

I am also grateful to the faculty and staff of the School of Information and Communications Technology at Hanoi University of Science and Technology for providing a stimulating academic environment and the necessary resources for this research.

I would like to thank my family for their unwavering encouragement and understanding during the long hours spent on this project. Their belief in me has been a constant source of motivation.

Lastly, I extend my appreciation to the open-source community for the tools and libraries that made this project possible, including Scapy, XGBoost, FastAPI, React, and many others.

ABSTRACT

The rapid adoption of DNS-over-HTTPS (DoH) has significantly enhanced user privacy by encrypting DNS queries within HTTPS traffic. However, this encryption also creates a security gap: malware can abuse DoH to hide command-and-control communication from traditional network security tools that rely on inspecting plaintext DNS traffic. This graduation thesis presents ShieldX, an intelligent endpoint security agent that detects malware communication over DoH using machine learning. The system consists of two main components: a lightweight Python-based endpoint agent and a centralized web management dashboard. The agent captures live TLSv1.3 traffic, extracts 28 statistical features from bidirectional network flows (including packet length distributions, inter-arrival times, response time statistics, and byte-level metrics), and classifies flows using a pre-trained XGBoost model. The model, trained on the CIC-DoHBrw-2020 dataset of 499,106 labeled flows, achieves 99.2% accuracy. The web dashboard, built with FastAPI and React, provides security administrators with centralized visibility into agent status, malware alerts, and domain whitelist management. The system also integrates with the Linux nftables firewall for kernel-level enforcement of domain whitelists. ShieldX bridges the gap between academic research on encrypted traffic analysis and practical, deployable endpoint security, offering a complete pipeline from real-time packet capture to centralized alert management.

Student

(Signature and full name)

TABLE OF CONTENTS

CHAPTER 1. INTRODUCTION.....	1
1.1 Motivation	1
1.2 Objectives and scope of the graduation thesis	2
1.3 Tentative solution	3
1.4 Thesis organization.....	4
CHAPTER 2. REQUIREMENT SURVEY AND ANALYSIS.....	5
2.1 Status survey	5
2.1.1 Existing Solutions for DoH-based Malware Detection	5
2.1.2 Existing Security Solutions and Their Limitations.....	6
2.1.3 User and System Requirements Analysis	6
2.2 Functional Overview	7
2.2.1 General Use Case Diagram	7
2.2.2 Detailed Use Case Diagram	7
2.2.3 Business Process.....	7
2.3 Functional description.....	9
2.3.1 Description of use case “Heartbeat Registration”	9
2.3.2 Description of use case “Malware Alert Reporting”	10
2.3.3 Description of use case “Whitelist Domain Management”.....	10
2.3.4 Description of use case “Agent Status Monitoring”	11
2.3.5 Description of use case “System Health Dashboard”	11
2.3.6 Description of use case “Flow Detail Inspection”	12
2.4 Hands-on Implementation Considerations.....	13
2.4.1 Packet Capture Requirements	13
2.4.2 ML Model Deployment Strategy	13

2.4.3 Agent-Backend Communication Reliability	14
2.4.4 Security and Compliance	14
2.5 Non-functional requirement	14
CHAPTER 3. THEORETICAL BACKGROUND AND TECHNOLOGIES	16
3.1 DNS-over-HTTPS.....	16
3.1.1 DNS Protocol Overview	16
3.1.2 The Emergence of DoH	16
3.1.3 Security Implications of DoH.....	17
3.2 Statistical Network Flow Analysis	17
3.2.1 Network Flow Features	18
3.3 XGBoost.....	21
3.3.1 Gradient Boosting	21
3.3.2 XGBoost Algorithm	22
3.3.3 Why XGBoost for DoH Detection	22
3.4 Random Forest	23
3.4.1 Algorithm Overview	23
3.4.2 Mathematical Foundation	23
3.4.3 Advantages for DoH Detection	23
3.5 LightGBM	24
3.5.1 Algorithm Overview	24
3.5.2 Key Innovations	24
3.5.3 Leaf-wise Tree Growth	24
3.5.4 Advantages for DoH Detection	24
3.6 Support Vector Machine.....	25
3.6.1 Algorithm Overview	25

3.6.2 Mathematical Formulation	25
3.6.3 Kernel Trick	25
3.6.4 Limitations for DoH Detection	25
3.7 Technologies	26
3.7.1 Python and Scapy.....	26
3.7.2 FastAPI.....	26
3.7.3 SQLAlchemy and MySQL	26
3.7.4 React and Chakra UI	26
3.7.5 nftables	26
3.7.6 Scikit-learn.....	27
3.7.7 Training Pipeline.....	27
3.7.8 Model Serialization Formats	27
3.7.9 CIC-DoHBrw-2020 Dataset.....	28
CHAPTER 4. DESIGN, IMPLEMENTATION, AND EVALUATION.....	29
4.1 Architecture design.....	29
4.1.1 Software architecture selection	29
4.1.2 Overall design.....	30
4.1.3 Detailed package design	34
4.2 Detailed design.....	36
4.2.1 User interface design.....	36
4.2.2 Layer design	38
4.2.3 Database design	39
4.3 Application Building.....	40
4.3.1 Libraries and Tools.....	40
4.3.2 Achievement.....	40
4.3.3 Illustration of main functions	41

4.4 Testing.....	41
4.4.1 Agent Testing.....	41
4.4.2 Backend API Testing	42
4.4.3 Test Results	42
4.5 Deployment	42
4.5.1 Experimental Results.....	43
4.6 Model Comparison and Evaluation.....	44
4.6.1 Experimental Setup	44
4.6.2 Model Configuration	44
4.6.3 Quantitative Results	45
4.6.4 Detailed Error Analysis	45
4.6.5 Computational Efficiency	45
4.6.6 Discussion and Model Selection	46
4.6.7 Feature Importance Analysis	46
CHAPTER 5. SOLUTION AND CONTRIBUTION	48
5.1 Integrated DoH Malware Detection Pipeline	48
5.1.1 Problem	48
5.1.2 Solution	48
5.1.3 Implementation Details.....	49
5.1.4 Result	50
5.2 Dual-Component Agent-Dashboard Architecture	50
5.2.1 Problem	50
5.2.2 Solution	50
5.2.3 Result	53
5.3 nftables-based Domain Whitelist Enforcement	53
5.3.1 Problem	53

5.3.2 Solution	53
5.3.3 Result	54
CHAPTER 6. CONCLUSION AND FUTURE WORK	55
6.1 Conclusion.....	55
6.2 Limitations.....	56
6.3 Future Work.....	57
6.3.1 Short-term Improvements	57
6.3.2 Medium-term Extensions.....	58
6.3.3 Long-term Vision.....	58
REFERENCE	60

LIST OF FIGURES

Figure 2.1	General use case diagram of the ShieldX system	7
Figure 2.2	Activity diagram of the malware detection pipeline	9
Figure 3.1	DNS-over-HTTPS architecture showing the comparison between traditional DNS and encrypted DoH queries	17
Figure 3.2	Feature extraction pipeline: from raw packets to 28 statistical features per flow	21
Figure 4.1	High-level system architecture of ShieldX	29
Figure 4.2	Agent core layer: entry point, orchestrator, scheduler, domain modules, and supporting utilities	31
Figure 4.3	Packet processing layer: flow session management, flow objects, feature extractors, time-series analysis, and support modules	32
Figure 4.4	ML model training pipeline: from dataset preparation to trained model export and production deployment	33
Figure 4.5	Package dependency diagram showing module relationships across the ShieldX system	34
Figure 4.6	Class diagram of the ShieldX agent packet processing package	35
Figure 4.7	Dashboard page showing summary statistics	37
Figure 4.8	Agents management page showing registered agents and their status	37
Figure 4.9	Malware alerts page showing detected threats	37
Figure 4.10	Whitelist domain management page	38
Figure 4.11	Sequence diagram of the malware detection pipeline	39
Figure 4.12	Entity-Relationship diagram of the ShieldX database	40
Figure 4.13	Deployment diagram of the ShieldX system	43
Figure 5.1	Detailed communication flow between ShieldX Agent, Backend API, Database, and Frontend Dashboard, covering registration, heartbeat, whitelist sync, malware alerting, and admin operations	52

LIST OF TABLES

Bảng 3.1	Complete list of 28 statistical features extracted per flow . . .	21
Bảng 3.2	Summary of the CIC-DoHBrw-2020 dataset	28
Bảng 4.1	Libraries and tools used in ShieldX development	40
Bảng 4.2	Summary of test results	42
Bảng 4.3	XGBoost model performance on the CIC-DoHBrw-2020 test set	43
Bảng 4.4	Performance comparison of the four trained models on the CIC-DoHBrw-2020 test set (99,822 flows). Metrics show Mali- cious class scores.	45
Bảng 4.5	Confusion matrix comparison on 99,822 test samples	45
Bảng 4.6	Top 10 most important features according to the XGBoost model	47

LIST OF ABBREVIATIONS

Abriviation	Full Expression
API	Application Programming Interface
C2	Command and Control
CIC	Canadian Institute for Cybersecurity
DNS	Domain Name System
DoH	DNS-over-HTTPS
HTTPS	Hypertext Transfer Protocol Secure
IDS	Intrusion Detection System
ML	Machine Learning
ORM	Object-Relational Mapping
REST	Representational State Transfer
RFC	Request for Comments
SIEM	Security Information and Event Management
SPA	Single-Page Application
TLS	Transport Layer Security
XGBoost	Extreme Gradient Boosting

CHAPTER 1. INTRODUCTION

1.1 Motivation

The rapid expansion of the Internet and the increasing sophistication of cyber threats have made network security one of the most critical challenges of the digital age. Malware, ransomware, phishing, and data exfiltration attacks continue to evolve, exploiting weaknesses in traditional security mechanisms. According to recent cybersecurity reports, thousands of new malware variants are discovered daily, and organizations worldwide face substantial financial and reputational damage from security breaches. The annual cost of cybercrime is projected to reach trillions of dollars, making effective security solutions an urgent priority for businesses, governments, and individuals alike.

Domain Name System (DNS) is a fundamental protocol of the Internet, responsible for translating human-readable domain names (e.g., `www.example.com`) into machine-readable IP addresses. Due to its critical role and historically plaintext nature, DNS traffic has long been a vector for cyber attacks, including DNS tunneling, data exfiltration, and command-and-control (C2) communication. Malware authors frequently abuse DNS for C2 communication because DNS traffic is almost always allowed through firewalls and is rarely inspected for malicious content.

To address the privacy concerns of plaintext DNS, DNS-over-HTTPS (DoH) was introduced as a standard that encrypts DNS queries within HTTPS traffic. Defined in RFC 8484, DoH operates over TCP port 443 and uses HTTP/2 or HTTP/3 protocols to encapsulate DNS messages. While DoH significantly enhances user privacy and prevents eavesdropping by network intermediaries, it also introduces new security challenges. Malicious actors can abuse DoH to hide their DNS queries among legitimate encrypted traffic, bypassing traditional network security appliances that rely on inspecting plaintext DNS traffic. The encrypted nature of DoH means that security tools cannot distinguish between a benign DNS query for a popular website and a malicious query to a C2 server based on payload inspection alone.

Traditional signature-based intrusion detection systems (IDS) and firewall rules are often insufficient to detect malware communication over DoH because the DNS queries themselves are encrypted. The only observable patterns are statistical features of the network flows, such as packet lengths, inter-arrival times, and byte rates. This makes the problem well-suited for machine learning (ML) approaches, which can learn to distinguish between benign and malicious encrypted traffic pat-

terns based on statistical flow characteristics. Machine learning offers the advantage of adapting to new and evolving threats without requiring manual signature updates.

Several research efforts have applied machine learning to the detection of DoH traffic. The CIC-DoHBrw-2020 dataset, published by the Canadian Institute for Cybersecurity, provides a comprehensive collection of labeled benign and malicious DoH traffic flows. Researchers have explored various classifiers, including Random Forest, Support Vector Machines (SVM), XGBoost, LightGBM, and deep learning models, achieving high accuracy in controlled environments. However, most of these solutions remain at the research stage and have not been integrated into a deployable, real-time security agent that can operate on endpoint devices.

Given the gap between academic research and practical deployment, there is a clear need for a lightweight, cross-platform security agent that can (i) capture live network traffic, (ii) extract statistical features from encrypted flows, (iii) apply a pre-trained machine learning model to detect malicious DoH traffic, and (iv) provide a centralized management interface for security administrators. Such a system would bridge the gap between theoretical ML research and operational cybersecurity, making advanced threat detection accessible in real-world environments.

This graduation thesis addresses this need by designing and implementing ShieldX, an intelligent endpoint security agent for malware detection over DNS-over-HTTPS using machine learning. ShieldX combines packet capture, statistical feature extraction, XGBoost-based classification, and a centralized web dashboard into an integrated security solution. The system is designed to be practical, deployable, and maintainable, with a focus on real-world usability rather than theoretical accuracy alone.

1.2 Objectives and scope of the graduation thesis

The primary objective of this thesis is to design, implement, and evaluate a complete system for detecting malware communication over DNS-over-HTTPS using machine learning techniques. The system, named ShieldX, consists of two main components: a lightweight endpoint security agent and a centralized web-based management dashboard.

The specific objectives are as follows:

To study the theoretical background of DNS-over-HTTPS, network flow analysis, and machine learning classification techniques relevant to encrypted traffic analysis.

To survey existing solutions and research for DoH-based malware detection and identify their limitations in terms of real-time deployment and practical usability.

To design and implement the ShieldX endpoint agent capable of capturing live TLSv1.3 traffic, extracting 28 statistical features from network flows, and classifying them using a pre-trained XGBoost model.

To design and implement the ShieldX-web centralized dashboard for agent management, malware alert visualization, and domain whitelist administration.

To evaluate the performance of the XGBoost model on the CIC-DoHBrw-2020 dataset and validate the end-to-end system functionality.

The scope of this thesis encompasses the complete software development life cycle, from requirements analysis and system design to implementation, testing, and deployment. The system is designed for Linux-based endpoints (with extensibility to Windows) and utilizes a MySQL-backed FastAPI backend with a React-based frontend.

1.3 Tentative solution

To achieve the stated objectives, this thesis proposes a two-tier system architecture consisting of an endpoint security agent and a centralized web dashboard.

The ShieldX agent is developed in Python and employs Scapy for real-time packet capture on network interfaces. Captured TLSv1.3 packets are grouped into bidirectional flows based on source and destination IP addresses and ports. For each flow, 28 statistical features are extracted, covering packet length statistics, inter-arrival time statistics, response time statistics, and byte-level metrics. These features are fed into an XGBoost classifier that has been trained on the CIC-DoHBrw-2020 dataset. Detected malicious flows trigger alerts that are sent to the central backend via HTTP.

The ShieldX-web dashboard provides a FastAPI REST backend with three core modules: agent heartbeat monitoring, malware alert management, and domain whitelist CRUD operations. A React-based frontend with Chakra UI components offers a responsive and user-friendly interface for security administrators.

The main contributions of this thesis are as follows:

A complete, deployable pipeline for real-time DoH malware detection using statistical flow analysis and XGBoost classification.

A dual-component architecture that separates detection (agent) from management (dashboard), enabling scalable multi-agent deployments.

A domain whitelist mechanism with nftables integration that enforces allowlist-based network access control on endpoints.

A public web dashboard providing centralized visibility into agent status, security alerts, and whitelist configuration.

1.4 Thesis organization

The remainder of this graduation thesis is organized as follows.

Chapter 2 presents a comprehensive survey of the requirements for the ShieldX system. It analyzes existing solutions for DoH-based malware detection, presents the functional and non-functional requirements, and describes the use case model that drives the system design.

Chapter 3 introduces the theoretical background and technologies underpinning the ShieldX system. It covers DNS-over-HTTPS protocol fundamentals, statistical network flow analysis, the XGBoost machine learning algorithm, and the software tools and frameworks used in the implementation.

Chapter 4 details the design, implementation, and evaluation of both the ShieldX agent and the ShieldX-web dashboard. It presents the software architecture, detailed component design, database design, user interface design, implementation details, testing methodology, and deployment considerations.

Chapter 5 discusses the key solutions and contributions of this thesis, including the integrated detection pipeline, the agent-dashboard architecture, and the domain whitelist enforcement mechanism.

Chapter 6 concludes the thesis by summarizing the achievements, discussing limitations, and outlining directions for future work.

CHAPTER 2. REQUIREMENT SURVEY AND ANALYSIS

This chapter presents a comprehensive survey of the requirements for the ShieldX system. It begins by examining the current landscape of DoH-based malware detection and existing security solutions, then derives the functional and non-functional requirements that guide the system design. The use case model is presented to capture the interactions between users and the system.

2.1 Status survey

2.1.1 Existing Solutions for DoH-based Malware Detection

The detection of malware communication over DNS-over-HTTPS has attracted significant research interest in recent years. The CIC-DoHBrw-2020 dataset, published by the Canadian Institute for Cybersecurity (CIC), has become a benchmark for evaluating DoH traffic classification approaches. This dataset comprises over 500,000 labeled network flows with 28 statistical features, covering both benign and malicious DoH traffic scenarios.

Several machine learning approaches have been proposed for classifying DoH traffic:

MontazeriShatoori et al. [1] introduced the CIC-DoHBrw-2020 dataset and evaluated multiple classifiers including Random Forest, K-Nearest Neighbors, and Neural Networks, achieving over 99% accuracy in controlled experiments.

Singh and Kaur [2] proposed a hybrid approach combining feature selection with ensemble learning for DoH traffic classification, demonstrating that a reduced feature set could maintain high detection accuracy while reducing computational overhead.

Zebari et al. [3] applied deep learning architectures, including convolutional neural networks (CNNs) and long short-term memory (LSTM) networks, to the problem of encrypted DNS traffic analysis, achieving competitive results.

Despite the high accuracy reported in these studies, most existing solutions remain at the research stage and exhibit several common limitations:

They focus primarily on offline classification using pre-collected datasets rather than real-time packet capture.

They lack integration with a deployable agent that can operate on endpoint devices.

They do not provide a centralized management interface for monitoring and

configuration.

They do not incorporate network-level enforcement mechanisms such as fire-wall rules.

2.1.2 Existing Security Solutions and Their Limitations

Several commercial and open-source security tools address aspects of the DoH detection problem:

Splunk and **ELK Stack** provide powerful log analysis and SIEM capabilities but require significant configuration and lack native DoH-specific detection models. **Snort** and **Suricata** are network intrusion detection systems that can perform deep packet inspection, but their signature-based approach is ineffective against encrypted DoH traffic where the payload is not visible. **Wireshark** and **tcpdump** offer packet capture and analysis but are passive analysis tools without automated detection capabilities. **Cisco Umbrella** and **Cloudflare Gateway** provide DNS-layer security but operate as cloud services rather than endpoint agents and may introduce latency or privacy concerns.

The analysis of existing solutions reveals a gap in the market for a lightweight, open-source, endpoint-based DoH detection system with centralized management. ShieldX aims to fill this gap by providing an integrated solution that combines real-time packet capture, machine learning classification, and a web-based management dashboard.

2.1.3 User and System Requirements Analysis

The primary users of the ShieldX system are security administrators who need to monitor and protect endpoints in their network. The key requirements identified through analysis of the problem domain are as follows:

Real-time detection: The system must be capable of capturing and analyzing live network traffic to detect malicious DoH flows as they occur.

Accuracy: The detection model must achieve high accuracy with low false-positive rates to avoid alert fatigue.

Centralized visibility: Administrators must be able to view the status of all deployed agents, review malware alerts, and manage security policies from a single dashboard.

Network enforcement: The system should be able to enforce domain whitelists at the network level, blocking traffic to unauthorized domains.

Cross-platform compatibility: The agent should support both Linux and

terface. Packets are grouped into bidirectional flows, and 28 statistical features are extracted for each flow. The XGBoost model classifies the flow as benign or malicious. If malicious, an alert is sent to the backend, and the administrator can review the alert and take appropriate action, such as updating the whitelist or investigating the affected endpoint.

The complete business process can be described through the following sequence of activities:

1. **Agent Initialization:** The ShieldX agent starts and performs self-configuration, loading the XGBoost model file, initializing the packet capture interface, and registering with the backend via a heartbeat request.
2. **Network Monitoring:** The agent begins capturing packets on the configured network interface, filtering for TCP traffic on port 443 (HTTPS). Captured packets are buffered and grouped into bidirectional flows based on the standard 5-tuple.
3. **Flow Completion:** Flows are marked as complete when either (a) no packets have been observed for 40 seconds, or (b) the flow has existed for more than 120 seconds. Completed flows are passed to the feature extraction stage.
4. **Feature Extraction:** For each completed flow, a Python module computes 28 statistical features covering bytes-level metrics, packet length distribution statistics, packet timing patterns, and response time analysis.
5. **ML Classification:** The feature vector is passed to the XGBoost classifier, which produces a binary prediction: benign (0) or malicious (1). The prediction probability is also recorded for confidence assessment.
6. **Alerting and Logging:** If the prediction is malicious, the agent logs the flow details locally and sends an alert to the ShieldX-web backend via a REST API call. The backend stores the alert in the database for later review.
7. **Administrator Review:** The security administrator accesses the ShieldX-web dashboard to review alerts, examine flow details, and make decisions about whitelist updates or further investigation.
8. **Whitelist Synchronization:** On the next heartbeat cycle, the agent retrieves the updated whitelist from the backend and reconfigures the nftables firewall rules to enforce the new domain access policy.

This process runs continuously, with the detection cycle repeating every 120 seconds and the heartbeat cycle every 60 seconds. The system is designed to oper-

ate autonomously, requiring administrator intervention only when malicious traffic is detected.

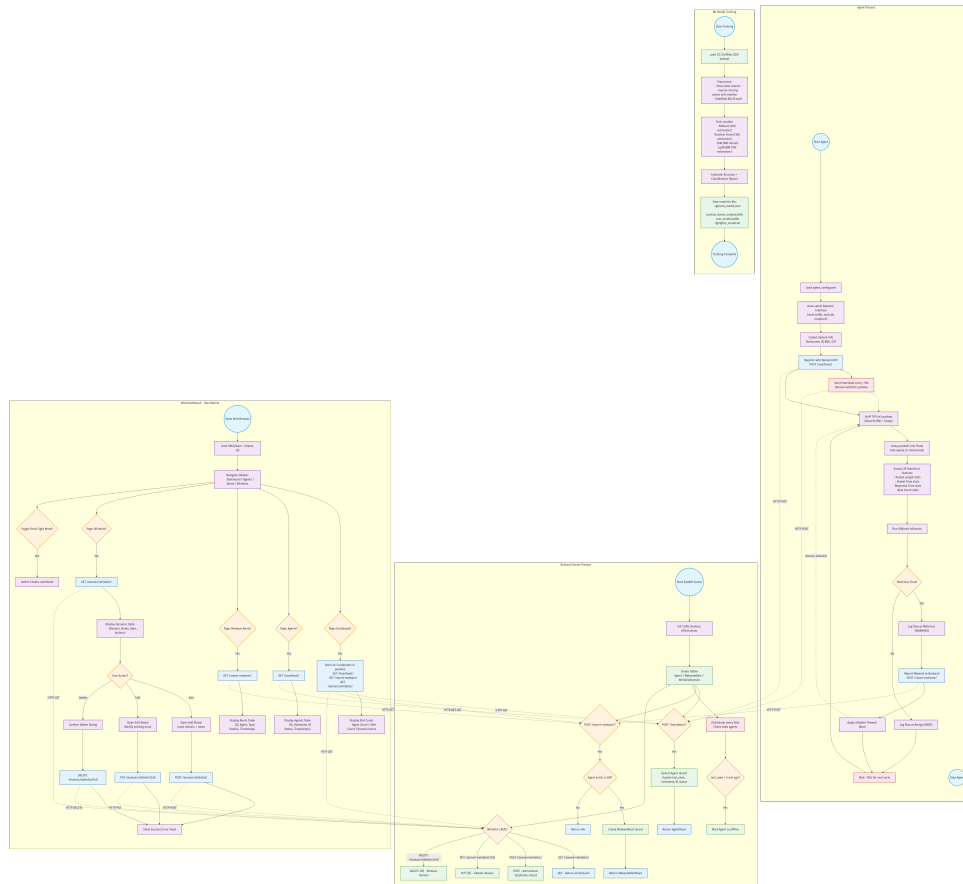


Figure 2.2: Activity diagram of the malware detection pipeline

2.3 Functional description

2.3.1 Description of use case “Heartbeat Registration”

Use case name: Heartbeat Registration

Primary actor: ShieldX Agent

Precondition: The agent is configured with the backend API URL and has network connectivity.

Flow of events:

1. The agent sends a POST request to /heartbeat/ with agent identification and system information.
2. The backend checks if the agent ID already exists in the database.
3. If the agent exists, the backend updates the last_seen timestamp and status to “active”.
4. If the agent is new, the backend creates a new agent record.

5. The backend returns a success response.

Postcondition: The agent is registered with an updated or newly created record in the database.

2.3.2 Description of use case “Malware Alert Reporting”

Use case name: Malware Alert Reporting

Primary actor: ShieldX Agent

Precondition: The agent has detected a malicious flow via the ML classifier.

Flow of events:

1. The agent extracts the 28 features from the malicious flow.
2. The agent sends a POST request to /report-malware/ with agent ID, malware type, and flow details.
3. The backend validates the agent ID and creates a new MalwareAlert record.
4. The backend returns the created alert details.

Alternative flow: If the agent ID is not found, the backend returns a 404 error.

Postcondition: A new malware alert is stored in the database and visible on the dashboard.

2.3.3 Description of use case “Whitelist Domain Management”

Use case name: Whitelist Domain Management

Primary actor: Security Administrator

Precondition: The administrator is logged into the ShieldX-web dashboard.

Main flow – Add domain:

1. The administrator navigates to the Whitelist page.
2. The administrator clicks the “Add Domain” button.
3. A modal dialog appears with fields for domain name and notes.
4. The administrator enters the domain and clicks “Add”.
5. The backend validates the domain and creates a new WhitelistDomain record.
6. The dashboard updates to show the new domain in the table.

Alternative flow: If the domain already exists, the backend returns a 400 error.

Exception flow: The administrator may edit or delete existing domains through similar modal dialogs.

Postcondition: The whitelist is updated, and the change is propagated to agents on their next heartbeat.

2.3.4 Description of use case “Agent Status Monitoring”

Use case name: Agent Status Monitoring

Primary actor: Security Administrator

Precondition: The administrator is logged into the ShieldX-web dashboard and at least one agent has registered with the backend.

Flow of events:

1. The administrator navigates to the Agents page on the dashboard.
2. The frontend sends a GET request to `/api/v1/agents/` to retrieve all registered agents.
3. The backend queries the database for all agent records and returns them as a JSON array.
4. The dashboard renders an interactive table showing agent ID, hostname, IP address, status (active/offline), last seen timestamp, and uptime.
5. Agents with a last_seen timestamp older than 120 seconds are displayed with a red “offline” indicator.
6. The table updates automatically every 30 seconds via a polling mechanism.

Alternative flow: If no agents are registered, the dashboard displays an empty state with instructions for deploying the agent.

Postcondition: The administrator has real-time visibility into all deployed agents and their operational status.

2.3.5 Description of use case “System Health Dashboard”

Use case name: System Health Dashboard

Primary actor: Security Administrator

Precondition: The administrator is logged into the ShieldX-web dashboard.

Flow of events:

1. The administrator navigates to the Dashboard page.

2. The frontend sends parallel GET requests to multiple API endpoints to collect statistics: total agent count, active agent count, total alerts, alerts in the last 24 hours, and whitelist domain count.
3. The backend aggregates and returns the statistics as a single response payload.
4. The dashboard renders summary cards with count metrics, each with an icon and color-coded background.
5. The administrator can click on each card to navigate to the corresponding detail page.

Postcondition: The administrator has an at-a-glance overview of the entire ShieldX deployment's health and security posture.

2.3.6 Description of use case “Flow Detail Inspection”

Use case name: Flow Detail Inspection

Primary actor: Security Administrator

Precondition: At least one malware alert has been reported to the backend by an agent.

Flow of events:

1. The administrator navigates to the Malware Alerts page on the dashboard.
2. The frontend sends a GET request to `/api/v1/alerts/` with optional query parameters for filtering (date range, agent ID, malware type).
3. The backend queries the MalwareAlert table with the applied filters and returns paginated results.
4. The dashboard renders an interactive table with columns for timestamp, agent hostname, malware type, flow source/destination IPs, and confidence score.
5. The administrator clicks on a specific alert row to expand its details, revealing the full 28-feature vector and packet-level metadata.
6. The administrator can export the alert details as a JSON file for further analysis or integration with external SIEM tools.

Alternative flow: If no alerts match the applied filters, an empty state message is displayed.

Postcondition: The administrator has inspected the details of a specific mali-

cious flow and can make informed decisions about further action.

2.4 Hands-on Implementation Considerations

Beyond the functional and non-functional requirements, several practical considerations inform the implementation of the ShieldX system:

2.4.1 Packet Capture Requirements

Real-time packet capture on endpoint devices requires careful consideration of operating system permissions and network interface characteristics:

Root/Sudo Privileges: Raw socket access for packet capture requires elevated privileges. The agent must be run as root or with `CAP_NET_RAW` capability on Linux.

Network Interface Selection: The agent must allow configuration of the target network interface. On multi-homed systems, traffic may need to be monitored on multiple interfaces.

Packet Filtering Efficiency: To reduce processing overhead, the agent uses BPF (Berkeley Packet Filter) expressions to capture only TCP traffic on port 443, discarding irrelevant packets at the kernel level before they reach the agent.

Buffer Management: High-traffic environments may generate thousands of packets per second. The agent must implement efficient buffer management to avoid packet drops while maintaining classification throughput.

2.4.2 ML Model Deployment Strategy

Deploying a trained machine learning model on endpoint devices presents unique challenges:

Model Portability: The trained model must be serialized in a format that can be loaded on any endpoint regardless of the training environment. XGBoost's JSON format provides this portability.

Version Management: As models are updated and improved, the agent must support model versioning and hot-swapping without requiring a full application restart.

Inference Latency: The classification step must complete within milliseconds to avoid becoming a bottleneck in the detection pipeline. This requires efficient model evaluation optimized for the target hardware.

Memory Footprint: The model must fit within the available memory of the endpoint device. Trees ensemble models can be compressed through pruning

or quantization when necessary.

2.4.3 Agent-Backend Communication Reliability

The agent operates independently but relies on the backend for centralized management:

Offline Resilience: The agent must continue detection and local logging even when the backend is unreachable. Alerts should be queued locally and transmitted when connectivity is restored.

Heartbeat Mechanism: Regular heartbeat signals indicate agent liveness. If no heartbeat is received within a configurable timeout, the backend should mark the agent as offline.

Idempotent Operations: API operations should be designed to be idempotent, allowing the agent to retry failed requests without creating duplicate records.

Backpressure Handling: If the backend is overwhelmed, it should signal the agent to reduce the alert frequency rather than dropping alerts.

2.4.4 Security and Compliance

The ShieldX system processes potentially sensitive network traffic data:

Data Privacy: The agent extracts only statistical features from network flows, not packet payloads. Statistical features cannot be reverse-engineered to recover original DNS queries, preserving user privacy.

Least Privilege: The agent operates with the minimum necessary privileges: raw socket access for packet capture and root access for nftables management. The backend requires only database write access for alerts and read access for the whitelist.

Audit Trail: All detection events are logged with timestamps and agent identification, providing a complete audit trail for security investigations.

Log Rotation: The agent implements log rotation to prevent disk exhaustion during prolonged operation. Old log files are compressed and archived according to configurable retention policies.

2.5 Non-functional requirement

The following non-functional requirements guide the design and implementation of the ShieldX system:

Performance: The agent must process and classify network flows in near real-

time. The capture-and-predict cycle should complete within 120 seconds to ensure timely detection. The ML model prediction for a single flow should complete in under 100 milliseconds.

Reliability: The agent must operate continuously without crashing. In the event of a backend connectivity failure, the agent should continue local detection and queue alerts for later transmission.

Scalability: The backend must support multiple agents (at least 50) reporting simultaneously. The API endpoints should respond within 500 milliseconds under normal load.

Usability: The web dashboard should be intuitive and responsive, requiring minimal training for security administrators. The UI should follow standard design patterns with clear navigation and visual feedback.

Maintainability: The codebase should follow modular design principles with clear separation of concerns. Each component should be independently testable and deployable.

Security: The system must validate all inputs at the API layer. The agent should enforce network whitelisting at the kernel level using nftables. Communication between agents and the backend should be over HTTPS in production deployments.

Cross-platform: The agent should be designed to support both Linux and Windows operating systems. The ML model should be portable and platform-independent.

CHAPTER 3. THEORETICAL BACKGROUND AND TECHNOLOGIES

This chapter presents the theoretical background and technologies that underpin the ShieldX system. It covers the DNS-over-HTTPS protocol, statistical network flow analysis, the XGBoost machine learning algorithm, and the key software frameworks used in the implementation. The purpose of this chapter is to provide the reader with the essential knowledge required to understand the design decisions and implementation details presented in subsequent chapters.

3.1 DNS-over-HTTPS

3.1.1 DNS Protocol Overview

The Domain Name System (DNS) is a hierarchical naming system that translates human-readable domain names (e.g., `www.example.com`) into IP addresses. Traditional DNS operates over UDP port 53 and transmits queries and responses in plaintext, making it vulnerable to eavesdropping, spoofing, and manipulation.

3.1.2 The Emergence of DoH

DNS-over-HTTPS (DoH), defined in RFC 8484 [4], addresses the privacy limitations of traditional DNS by encapsulating DNS queries and responses within HTTPS traffic. DoH operates over TCP port 443 and uses the standard HTTP/2 or HTTP/3 protocols. This provides several benefits: (i) DNS queries are encrypted, preventing eavesdropping by network intermediaries; (ii) DoH traffic is indistinguishable from regular HTTPS traffic, making it difficult to block; and (iii) DNS queries benefit from the security guarantees of TLS, including server authentication and integrity protection.

Popular DoH providers include Cloudflare (1.1.1.1), Google (8.8.8.8), Quad9 (9.9.9.9), and OpenDNS (208.67.222.222). These services have made DoH widely accessible to end users.

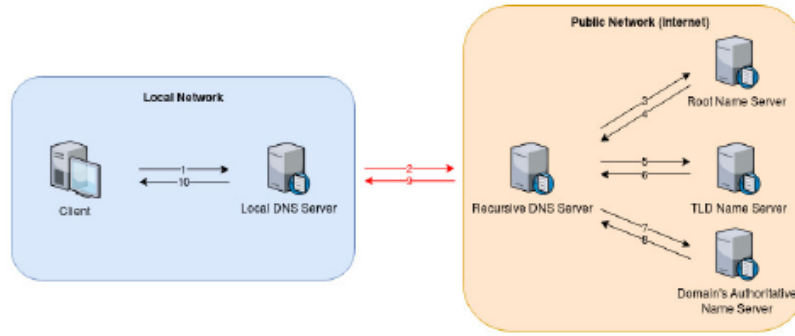


Fig. 1: Domain Name System [1]

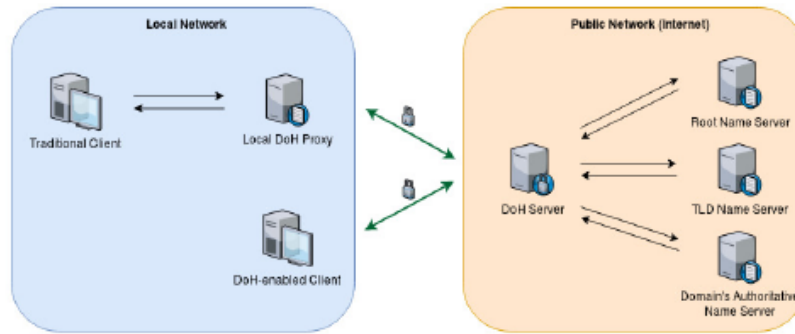


Fig. 2: DNS over HTTPS (DoH)

Figure 3.1: DNS-over-HTTPS architecture showing the comparison between traditional DNS and encrypted DoH queries

3.1.3 Security Implications of DoH

While DoH enhances privacy, it also introduces new security challenges. Malware authors can leverage DoH to hide C2 communication from network security tools. Traditional DNS monitoring solutions that rely on analyzing plaintext DNS queries become ineffective against DoH traffic. This creates a detection gap that adversaries can exploit. The only remaining observable characteristics are statistical properties of the network flows, such as packet sizes, timing patterns, and byte rates, which form the basis for the machine-learning approach adopted in this thesis.

3.2 Statistical Network Flow Analysis

Network flow analysis involves the examination of metadata extracted from sequences of packets sharing common properties (e.g., same source and destination IP addresses and ports, same protocol). In the context of encrypted traffic classification, where packet payloads are not accessible, statistical features derived from packet headers and timing provide the primary source of discriminative informa-

tion.

3.2.1 Network Flow Features

The ShieldX system extracts 28 statistical features from each bidirectional TCP flow. These features are grouped into four categories of eight measures each, except for the bytes-level category which has four measures. The following subsections present the mathematical formulation of each feature.

Notation: For a given flow, let $\{p_1, p_2, \dots, p_n\}$ be the sequence of packets in the flow. Each packet p_i has a payload length l_i , a timestamp t_i , and a direction $d_i \in \{\text{FORWARD}, \text{REVERSE}\}$. Let n_f be the number of forward packets and n_r the number of reverse packets.

a, Bytes-Level Features (4 features)

These features measure the volume and rate of data transmission in each direction:

$$\text{FlowBytesSent} = \sum_{i:d_i=\text{FORWARD}} l_i \quad (3.1)$$

$$\text{FlowSentRate} = \frac{\text{FlowBytesSent}}{\max(t_n - t_1, \epsilon)} \quad (3.2)$$

$$\text{FlowBytesReceived} = \sum_{i:d_i=\text{REVERSE}} l_i \quad (3.3)$$

$$\text{FlowReceivedRate} = \frac{\text{FlowBytesReceived}}{\max(t_n - t_1, \epsilon)} \quad (3.4)$$

where ϵ is a small positive constant to prevent division by zero. These features capture the asymmetry between upstream and downstream traffic, which is often characteristic of malware C2 communication where small queries generate large responses or vice versa.

b, Packet Length Features (8 features)

Let $\{l_1, l_2, \dots, l_n\}$ be the sequence of packet payload lengths in the flow. The following eight statistical measures are computed:

$$\mu_l = \frac{1}{n} \sum_{i=1}^n l_i \quad (\text{Mean}) \quad (3.5)$$

$$\sigma_l^2 = \frac{1}{n} \sum_{i=1}^n (l_i - \mu_l)^2 \quad (\text{Variance}) \quad (3.6)$$

$$\sigma_l = \sqrt{\sigma_l^2} \quad (\text{Standard Deviation}) \quad (3.7)$$

$$\tilde{l} = \text{median of } \{l_1, l_2, \dots, l_n\} \quad (\text{Median}) \quad (3.8)$$

$$\hat{l} = \text{mode of } \{l_1, l_2, \dots, l_n\} \quad (\text{Mode}) \quad (3.9)$$

$$\text{Skew}_{\text{median}} = \frac{3(\mu_l - \tilde{l})}{\sigma_l} \quad (\text{Skewness from Median}) \quad (3.10)$$

$$\text{Skew}_{\text{mode}} = \frac{\mu_l - \hat{l}}{\sigma_l} \quad (\text{Skewness from Mode}) \quad (3.11)$$

$$\text{CV}_l = \frac{\sigma_l}{\mu_l} \quad (\text{Coefficient of Variation}) \quad (3.12)$$

Malicious traffic typically exhibits different packet length distributions compared to benign traffic. C2 communication often uses small, regular heartbeat packets, while data exfiltration involves large, irregular packet bursts. The skewness measures capture the asymmetry of the distribution, which tends to differ between human-generated and automated traffic.

c, Packet Time Features (8 features)

Let $\{\Delta t_1, \Delta t_2, \dots, \Delta t_{n-1}\}$ be the sequence of inter-arrival times between consecutive packets, where $\Delta t_i = t_{i+1} - t_i$. The same eight statistical measures are computed:

$$\mu_{\Delta t} = \frac{1}{n-1} \sum_{i=1}^{n-1} \Delta t_i \quad (\text{Mean}) \quad (3.13)$$

$$\sigma_{\Delta t}^2 = \frac{1}{n-1} \sum_{i=1}^{n-1} (\Delta t_i - \mu_{\Delta t})^2 \quad (\text{Variance}) \quad (3.14)$$

$$\sigma_{\Delta t} = \sqrt{\sigma_{\Delta t}^2} \quad (\text{Standard Deviation}) \quad (3.15)$$

$$\widetilde{\Delta t} = \text{median of } \{\Delta t_1, \dots, \Delta t_{n-1}\} \quad (\text{Median}) \quad (3.16)$$

$$\widehat{\Delta t} = \text{mode of } \{\Delta t_1, \dots, \Delta t_{n-1}\} \quad (\text{Mode}) \quad (3.17)$$

$$\text{Skew}_{\Delta t}^{\text{median}} = \frac{3(\mu_{\Delta t} - \widetilde{\Delta t})}{\sigma_{\Delta t}} \quad (\text{Skewness from Median}) \quad (3.18)$$

$$\text{Skew}_{\Delta t}^{\text{mode}} = \frac{\mu_{\Delta t} - \widehat{\Delta t}}{\sigma_{\Delta t}} \quad (\text{Skewness from Mode}) \quad (3.19)$$

$$\text{CV}_{\Delta t} = \frac{\sigma_{\Delta t}}{\mu_{\Delta t}} \quad (\text{Coefficient of Variation}) \quad (3.20)$$

Timing patterns are particularly informative for distinguishing automated malware traffic from human-generated benign traffic. Malware C2 communication often follows regular, periodic patterns with low variance, while human web browsing produces bursty, irregular inter-arrival times.

d, Response Time Features (8 features)

Response time features capture the timing between forward-reverse packet pairs. For each forward packet p_i^f , let p_j^r be the next reverse packet that arrives after p_i^f . The response time is defined as $\Delta r_i = t_j^r - t_i^f$. Let $\{\Delta r_1, \Delta r_2, \dots, \Delta r_m\}$ be the sequence of response times:

$$\mu_{\Delta r} = \frac{1}{m} \sum_{i=1}^m \Delta r_i \quad (\text{Mean}) \quad (3.21)$$

$$\sigma_{\Delta r}^2 = \frac{1}{m} \sum_{i=1}^m (\Delta r_i - \mu_{\Delta r})^2 \quad (\text{Variance}) \quad (3.22)$$

$$\sigma_{\Delta r} = \sqrt{\sigma_{\Delta r}^2} \quad (\text{Standard Deviation}) \quad (3.23)$$

$$\widetilde{\Delta r} = \text{median of } \{\Delta r_1, \dots, \Delta r_m\} \quad (\text{Median}) \quad (3.24)$$

$$\widehat{\Delta r} = \text{mode of } \{\Delta r_1, \dots, \Delta r_m\} \quad (\text{Mode}) \quad (3.25)$$

$$\text{Skew}_{\Delta r}^{\text{median}} = \frac{3(\mu_{\Delta r} - \widetilde{\Delta r})}{\sigma_{\Delta r}} \quad (\text{Skewness from Median}) \quad (3.26)$$

$$\text{Skew}_{\Delta r}^{\text{mode}} = \frac{\mu_{\Delta r} - \widehat{\Delta r}}{\sigma_{\Delta r}} \quad (\text{Skewness from Mode}) \quad (3.27)$$

$$\text{CV}_{\Delta r} = \frac{\sigma_{\Delta r}}{\mu_{\Delta r}} \quad (\text{Coefficient of Variation}) \quad (3.28)$$

Response time features capture server-side behavior patterns. Benign DNS servers typically respond quickly and consistently, while malicious C2 servers may exhibit more variable response times due to additional processing or network evasion techniques.

e, Feature Summary

The selection of these 28 features is based on the CIC-DoHBrw-2020 dataset [1] and prior research demonstrating their effectiveness for encrypted traffic classification. The complete list of all 28 features is summarized in the following table:

Category	Feature Names	Count
Bytes-Level	FlowBytesSent, FlowSentRate, FlowBytesReceived, FlowReceivedRate	4
Packet Length	PacketLengthVariance, PacketLengthStandardDeviation, PacketLengthMean, PacketLengthMedian, PacketLengthMode, PacketLengthSkewFromMedian, PacketLengthSkewFromMode, PacketLengthCoefficientofVariation	8
Packet Time	PacketTimeVariance, PacketTimeStandardDeviation, PacketTimeMean, PacketTimeMedian, PacketTimeMode, PacketTimeSkewFromMedian, PacketTimeSkewFromMode, PacketTimeCoefficientofVariation	8
Response Time	ResponseTimeTimeVariance, ResponseTimeTimeStandardDeviation, ResponseTimeTimeMean, ResponseTimeTimeMedian, ResponseTimeTimeMode, ResponseTimeTimeSkewFromMedian, ResponseTimeTimeSkewFromMode, ResponseTimeTimeCoefficientofVariation	8
Total		28

Table 3.1: Complete list of 28 statistical features extracted per flowTABLE I
STATISTICAL FEATURES

Parameter	Feature
F1	Number of flow bytes sent
F2	Rate of flow bytes sent
F3	Number of flow bytes received
F4	Rate of flow bytes received
F5-F12	Packet Length (F5: Mean, F6: Median, F7: Mode, F8: Variance, F9: Standard deviation, F10: Coefficient of variation, F11: Skew from median, F12: Skew from mode)
F13-20	Packet Time (F13: Mean, F14: Median, F15: Mode, F16: Variance, F17: Standard deviation, F18: Coefficient of variation, F19: Skew from median, F20: Skew from mode)
F21-F28	Request/response time difference (F21: Mean, F22: Median, F23: Mode, F24: Variance, F25: Standard deviation, F26: Coefficient of variation, F27: Skew from median, F28: Skew from mode)

Figure 3.2: Feature extraction pipeline: from raw packets to 28 statistical features per flow

3.3 XGBoost

3.3.1 Gradient Boosting

Gradient boosting is an ensemble machine learning technique that builds a strong classifier by combining multiple weak learners (typically decision trees) in an additive manner. At each iteration, a new tree is trained to predict the residual errors of the current ensemble, effectively minimizing a differentiable loss function through

gradient descent.

3.3.2 XGBoost Algorithm

XGBoost (Extreme Gradient Boosting) [5] is an optimized implementation of gradient boosting that has become one of the most widely used algorithms in machine learning competitions and industrial applications. XGBoost offers several advantages over traditional gradient boosting:

Regularization: XGBoost includes L1 (Lasso) and L2 (Ridge) regularization in its objective function, which helps prevent overfitting and improves generalization.

Tree pruning: Unlike traditional gradient boosting that grows trees to a fixed depth and then prunes, XGBoost grows trees greedily and prunes them based on loss reduction, leading to more efficient trees.

Handling missing values: XGBoost learns default branch directions for missing values, eliminating the need for explicit imputation.

Parallelization: The algorithm is designed for efficient parallel computation, significantly reducing training time.

Cross-validation: Built-in cross-validation at each iteration eliminates the need for a separate cross-validation pipeline.

3.3.3 Why XGBoost for DoH Detection

XGBoost was selected as the primary classifier for ShieldX for several reasons:

It achieves state-of-the-art performance on tabular data, which is the format of the 28 statistical features.

Previous studies on the CIC-DoHBrw-2020 dataset have shown that gradient-boosted trees achieve accuracy comparable to or exceeding deep learning approaches while requiring significantly fewer computational resources.

The trained model is lightweight (a few megabytes) and portable, making it suitable for deployment on endpoint devices with limited resources.

The model produces interpretable feature importance scores, enabling analysis of which statistical features are most discriminative.

The ShieldX XGBoost model is configured with 200 trees (`n_estimators`) and a maximum depth of 6, which balances model capacity with inference speed.

3.4 Random Forest

3.4.1 Algorithm Overview

Random Forest [6] is an ensemble learning method that operates by constructing a multitude of decision trees at training time and outputting the class that is the mode of the classes (classification) of the individual trees. It belongs to the family of bagging (Bootstrap Aggregating) methods, where each tree is trained on a different bootstrap sample of the original dataset.

The algorithm introduces two sources of randomness: (i) each tree is trained on a different bootstrap sample (random sampling with replacement), and (ii) at each node split, only a random subset of features is considered for splitting. This dual randomization decorrelates the individual trees, reducing variance without substantially increasing bias.

3.4.2 Mathematical Foundation

For a given input vector $\mathbf{x}$, the Random Forest classifier produces a prediction by aggregating the outputs of B individual trees:

$$\hat{y} = \text{mode} \{T_b(\mathbf{x})\}_{b=1}^B \quad (3.29)$$

where $T_b(\mathbf{x})$ is the prediction of the b -th tree. Each tree is grown without pruning, and the Gini impurity is typically used as the splitting criterion:

$$G(t) = 1 - \sum_{i=1}^C p_i^2 \quad (3.30)$$

where p_i is the proportion of samples belonging to class i at node t , and C is the number of classes.

3.4.3 Advantages for DoH Detection

Random Forest offers several advantages for encrypted traffic classification: it handles high-dimensional feature spaces naturally, provides feature importance scores for interpretability, is robust to outliers and noisy data, and does not require extensive hyperparameter tuning. The out-of-bag (OOB) error estimate provides an unbiased internal validation metric without requiring a separate validation set.

3.5 LightGBM

3.5.1 Algorithm Overview

LightGBM [7] is a gradient boosting framework developed by Microsoft that uses tree-based learning algorithms. Unlike traditional gradient boosting decision tree (GBDT) implementations that grow trees by level (level-wise growth), LightGBM grows trees leaf-wise, meaning it chooses the leaf with the highest loss reduction to expand at each step.

3.5.2 Key Innovations

LightGBM introduces two key techniques to accelerate training while maintaining accuracy:

Gradient-based One-Side Sampling (GOSS): GOSS retains instances with large gradients (i.e., poorly predicted samples) while randomly discarding instances with small gradients. This focuses the training on the most informative samples, significantly reducing the number of data instances used for information gain estimation.

Exclusive Feature Bundling (EFB): EFB bundles mutually exclusive features (features that rarely take non-zero values simultaneously) to reduce the effective number of features. This is particularly effective for high-dimensional sparse data.

3.5.3 Leaf-wise Tree Growth

The leaf-wise growth strategy can be formalized as:

$$\Delta L = \frac{(\sum_{i \in I_L} g_i)^2}{|I_L| + \lambda} + \frac{(\sum_{i \in I_R} g_i)^2}{|I_R| + \lambda} - \frac{(\sum_{i \in I} g_i)^2}{|I| + \lambda} \quad (3.31)$$

where g_i is the first-order gradient of the loss function at sample i , I , I_L , I_R are the sets of indices at the current node, left child, and right child respectively, and λ is the regularization parameter. The leaf with the maximum ΔL is selected for splitting at each iteration.

3.5.4 Advantages for DoH Detection

LightGBM excels in scenarios with large datasets and high-dimensional features. For the CIC-DoHBrw-2020 dataset, LightGBM's histogram-based algorithm reduces memory consumption significantly, and its leaf-wise growth often converges faster than level-wise alternatives. The resulting model is extremely compact, making it ideal for deployment on endpoints with limited storage.

3.6 Support Vector Machine

3.6.1 Algorithm Overview

Support Vector Machine (SVM) [8] is a supervised learning model that finds the optimal hyperplane separating classes in a high-dimensional feature space. The fundamental idea is to maximize the margin between the decision boundary and the nearest training samples from each class (the support vectors).

3.6.2 Mathematical Formulation

For a binary classification problem with training data $\{(\mathbf{x}_i, y_i)\}_{i=1}^N$, where $y_i \in \{-1, +1\}$, SVM solves the following optimization problem:

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^N \xi_i \quad (3.32)$$

subject to the constraints:

$$y_i(\mathbf{w}^T \phi(\mathbf{x}_i) + b) \geq 1 - \xi_i, \quad \xi_i \geq 0, \quad \forall i \quad (3.33)$$

where $\mathbf{w}$ is the weight vector, b is the bias term, C is the regularization parameter controlling the trade-off between margin maximization and classification error, ξ_i are slack variables for non-separable data, and ϕ is the kernel function mapping inputs to a higher-dimensional space.

3.6.3 Kernel Trick

The kernel trick enables SVM to learn non-linear decision boundaries without explicitly computing the transformed feature vectors. The RBF (Radial Basis Function) kernel, used in the ShieldX SVM model, is defined as:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2) \quad (3.34)$$

where γ controls the influence of each training sample. The RBF kernel can handle non-linear relationships in the feature space and has fewer numerical difficulties than polynomial kernels.

3.6.4 Limitations for DoH Detection

SVM faces several challenges for large-scale DoH traffic classification: (i) training complexity scales quadratically with the number of samples, making it impractical for the full CIC-DoHBrw-2020 dataset; (ii) inference requires computing the kernel function with all support vectors, leading to slower predictions;

(iii) SVM does not naturally produce probability estimates without additional Platt scaling; and (iv) performance is sensitive to feature scaling, requiring careful pre-processing.

3.7 Technologies

3.7.1 Python and Scapy

Python was chosen as the primary development language for the ShieldX agent due to its extensive ecosystem of networking and machine learning libraries. Scapy [9] is a Python library for packet manipulation and capture that provides a powerful API for crafting, sending, receiving, and analyzing network packets. ShieldX uses Scapy's AsyncSniffer for non-blocking packet capture on network interfaces, enabling concurrent packet processing and classification.

3.7.2 FastAPI

FastAPI [10] is a modern, high-performance web framework for building APIs with Python. It leverages Python type hints for automatic request validation and documentation generation. In ShieldX, FastAPI serves as the backend framework for the central management API, providing endpoints for agent registration, heart-beat monitoring, malware alert reporting, and domain whitelist management.

3.7.3 SQLAlchemy and MySQL

SQLModel [11] is an Object-Relational Mapping (ORM) library that combines SQLAlchemy and Pydantic into a unified API. It allows defining database models as Python classes with automatic validation and serialization. ShieldX uses SQLModel with a MySQL database for persistent storage of agent records, malware alerts, and whitelist domains.

3.7.4 React and Chakra UI

React is a JavaScript library for building user interfaces, while Chakra UI provides a component library with a focus on accessibility and developer experience. ShieldX-web's frontend is built as a React single-page application (SPA) using Vite as the build tool. Chakra UI provides pre-built components for layout, forms, tables, and notifications, enabling rapid development of a professional-grade dashboard.

3.7.5 nftables

nftables is the successor to iptables in the Linux kernel, providing a framework for packet filtering and classification. ShieldX uses nftables to enforce domain whitelists on agent endpoints. When the whitelist is updated, the agent resolves whitelisted domains to IP addresses and creates nftables rules that accept traffic to those IPs while dropping all other outbound traffic.

3.7.6 Scikit-learn

Scikit-learn [12] is a comprehensive machine learning library for Python that provides unified APIs for classification, regression, clustering, dimensionality reduction, and model evaluation. In ShieldX’s model training pipeline, scikit-learn is used for: (i) data preprocessing and feature scaling via its `StandardScaler` and `LabelEncoder` utilities; (ii) stratified train-test splitting to preserve class distribution; (iii) training the Random Forest and SVM classifiers; and (iv) model evaluation through metrics such as accuracy, precision, recall, F1-score, ROC-AUC, and confusion matrices. The `joblib` library, which is part of the scikit-learn ecosystem, is used for serializing and deserializing trained models for deployment.

3.7.7 Training Pipeline

The ShieldX model training pipeline is orchestrated by `train_all_models.py`, which sequentially executes four independent training scripts, one for each algorithm. Each script follows a consistent workflow:

1. **Data Loading:** The CIC-DoHBrw-2020 dataset is loaded from CSV format using `pandas`.
2. **Preprocessing:** Missing numerical values are imputed with the median of each feature column. The feature matrix and label vector are separated.
3. **Splitting:** An 80/20 stratified train-test split ensures that the class distribution is preserved in both subsets.
4. **Training:** The model is trained on the training set with algorithm-specific hyperparameters.
5. **Evaluation:** Accuracy and a full classification report (precision, recall, F1-score per class) are computed on the held-out test set.
6. **Serialization:** The trained model is saved to disk in a platform-independent format for later deployment.

The pipeline is fully automated and reproducible: the fixed `random_state=42` ensures that the same train-test split is used across all models, enabling fair comparison. Each trained model file is stored at the ShieldX project root, from where the agent can load it at runtime.

3.7.8 Model Serialization Formats

Each machine learning framework uses its own serialization format for trained models:

joblib (Random Forest, SVM): Python’s `joblib` library provides efficient se-

rialization of large numpy arrays and scikit-learn estimators. The resulting files (`random_forest_model.joblib`, `svm_model.joblib`) can be loaded into any Python environment with compatible library versions.

LightGBM text format (`lightgbm_model.txt`): LightGBM’s native text format stores the tree structure, split conditions, and leaf values in a human-readable format. This enables cross-language deployment and inspection of the model structure.

XGBoost JSON format (`xgboost_model.json`): XGBoost’s JSON serialization stores the complete model including tree structures, base scores, and configuration parameters. The JSON format is language-agnostic and can be loaded by XGBoost libraries in Python, R, Java, C++, and other languages.

3.7.9 CIC-DoHBrw-2020 Dataset

The CIC-DoHBrw-2020 dataset [1] is the primary dataset used for training and evaluating the ShieldX detection model. It contains 499,106 labeled network flows with 28 statistical features, covering both benign and malicious DoH traffic scenarios. The malicious traffic includes various malware types such as ransomware, trojans, and botnets communicating via C2 channels over DoH.

Property	Value
Number of flows	499,106
Number of features	28
Classes	Benign, Malicious
Year published	2020
Source	Canadian Institute for Cybersecurity (CIC)

Table 3.2: Summary of the CIC-DoHBrw-2020 dataset

CHAPTER 4. DESIGN, IMPLEMENTATION, AND EVALUATION

This chapter presents the complete design, implementation, and evaluation of the ShieldX system. It begins with the architectural design of the overall system, followed by detailed design of both the endpoint agent and the web dashboard. The implementation details, including the technologies used, achieved results, and user interface illustrations, are then presented. Finally, the testing methodology, experimental results, and deployment considerations are discussed.

4.1 Architecture design

4.1.1 Software architecture selection

The ShieldX system adopts a **two-tier client-server architecture** with elements of the **microservices** pattern. The architecture separates the system into two major components: the endpoint agent (ShieldX Agent) and the centralized management server (ShieldX-web). This separation provides several advantages:

Scalability: Multiple agents can be deployed across different endpoints, all reporting to a single backend server.

Independent deployment: The agent and backend can be updated, tested, and deployed independently.

Centralized management: Security administrators gain unified visibility into all agents through the web dashboard.

Fault isolation: A failure in the backend does not affect agent operation (agents continue local detection and can queue alerts).

The agent follows a **pipeline architecture** internally, where packets flow through a sequence of processing stages: capture, flow assembly, feature extraction, classification, and alerting. The backend follows a **layered architecture** with separate layers for API routing, business logic, data access, and persistence.

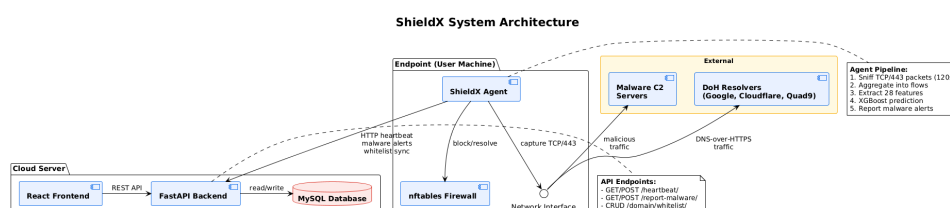


Figure 4.1: High-level system architecture of ShieldX

4.1.2 Overall design

The system comprises the following packages, organized into distinct architectural tiers:

shieldx-agent: The endpoint agent package containing sub-packages for packet processing (`packet_processing`), domain logic (`domains`), model inference (`models`), configuration (`config`), and the agent orchestrator.

shieldx-web-backend: The FastAPI backend package containing sub-packages for API routes (`api/v1/endpoints`), service layer (`services`), data models (`models`), middleware (`middleware`), and utility functions (`utils`).

shieldx-web-frontend: The React frontend package containing page components (`pages`), API client (`api.ts`), type definitions (`types.ts`), and theme configuration (`theme.ts`).

The ShieldX system is organized into multiple architectural layers, each with a distinct responsibility. The following subsections detail each layer's package structure and internal dependencies.

a, Frontend Layer (ShieldX-web Dashboard)

The frontend layer is a React single-page application (SPA) built with TypeScript and Chakra UI. It communicates with the backend exclusively through REST API calls via the Axios HTTP client. The layer is divided into four page components, a shared API client module, and TypeScript type definitions.

The frontend package (`shieldx-web-frontend`) follows a flat page-based structure: `pages/` contains four main views (Dashboard, Agents, MalwareAlerts, Whitelist), `api.ts` provides typed API functions via Axios, and `types.ts` defines TypeScript interfaces for the data models. The application is built with Vite and uses Chakra UI for responsive layout components.

b, Backend Layer (ShieldX-web API)

The backend layer implements a FastAPI REST API that follows a layered architecture pattern. The application factory in `app/main.py` configures middleware, registers routers, and manages the application lifecycle. Configuration is managed through Pydantic settings in `app/config.py`. The API layer (`api/v1/`) contains route handlers and dependency injection functions. The middleware layer provides request logging via a custom ASGI middleware. The models layer defines Pydantic schemas for request/response validation. The utils layer provides centralized logging configuration. The services layer contains reusable business logic, and the exceptions layer defines application-specific error types.

c, Agent Core Layer (ShieldX Agent)

The agent core layer is the heart of the ShieldX endpoint agent. It provides two execution modes: the full orchestrator (`agent_orchestrator.py`) that integrates heartbeat, whitelist management, and detection; and the lightweight scheduler (`agent_scheduler.py`) that runs only the detection pipeline on a timer.

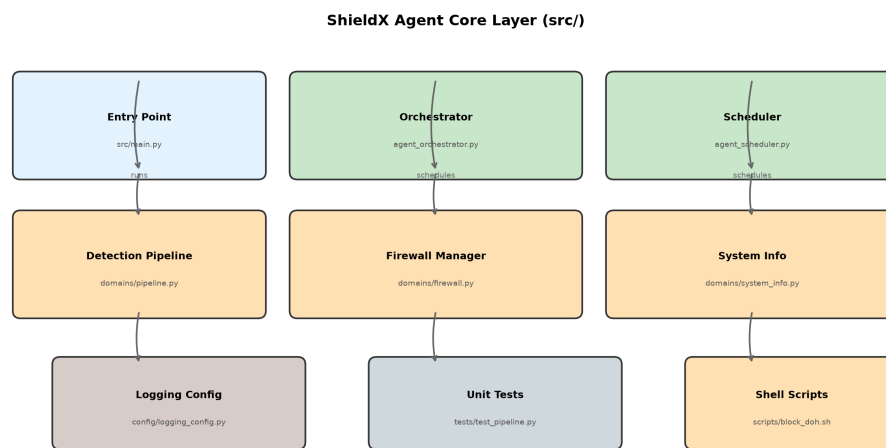


Figure 4.2: Agent core layer: entry point, orchestrator, scheduler, domain modules, and supporting utilities

The domain layer consists of three key modules: `pipeline.py` orchestrates the end-to-end detection flow (packet capture, feature extraction, ML inference), `firewall.py` manages nftables-based domain whitelist enforcement, and `system_info.py` collects endpoint system information for agent registration. The logging configuration is centralized in `config/logging_config.py`.

d, Packet Processing Layer

The packet processing layer is the most architecturally complex subsystem, responsible for transforming raw network packets into structured statistical features. It is organized into multiple sub-packages for separation of concerns.

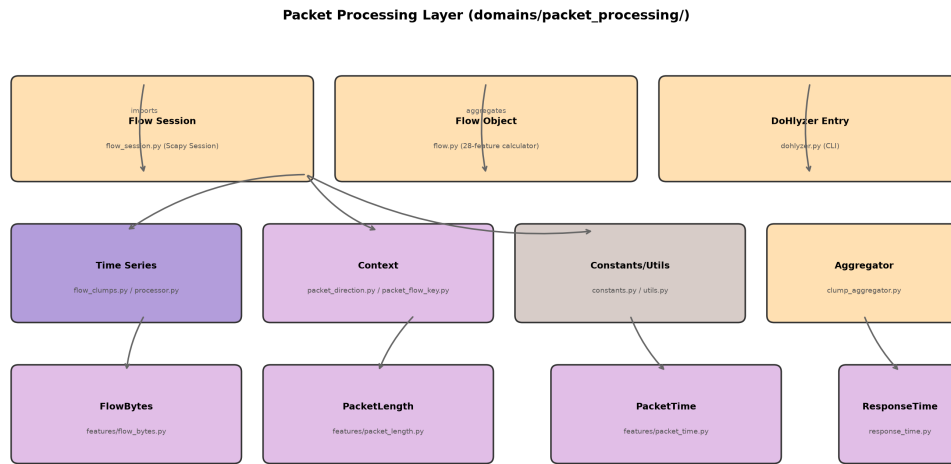


Figure 4.3: Packet processing layer: flow session management, flow objects, feature extractors, time-series analysis, and support modules

`flow_session.py` defines a custom Scapy session that intercepts packets, groups them into bidirectional flows, and manages flow lifecycle (creation, update, expiration, garbage collection). `flow.py` represents a single flow and provides the `get_data()` method that computes the 28 statistical features by delegating to four feature extractor classes: `FlowBytes`, `PacketLength`, `PacketTime`, and `ResponseTime`. The context sub-package (`features/context/`) provides foundational types such as `PacketDirection` and `PacketFlowKey` that are used across multiple feature extractors. The time-series sub-package provides an alternative clump-based flow representation for specialized analysis.

e, ML Model Training Layer

The model training layer is independent from the agent runtime and is used exclusively during development to train and evaluate the four machine learning models.

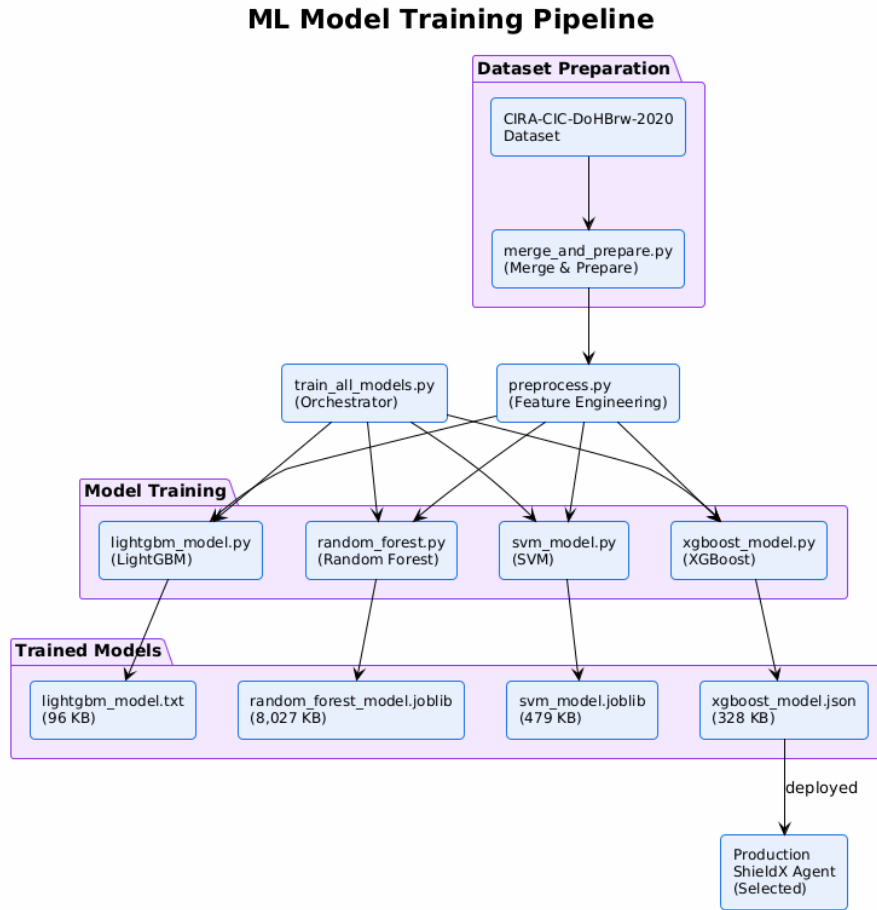


Figure 4.4: ML model training pipeline: from dataset preparation to trained model export and production deployment

The training pipeline is orchestrated by `train_all_models.py`, which spawns each model training script as a separate subprocess. All four scripts (`random_forest.py`, `lightgbm_model.py`, `xgboost_model.py`, `svm_model.py`) share the same data preprocessing module (`preprocess.py`) for loading and preparing the CIC-DoHBrw-2020 dataset. The `merge_and_prepare.py` script handles initial data merging of benign and malicious CSV files.

f, External Libraries and Frameworks

The ShieldX system depends on a rich ecosystem of open-source libraries and frameworks that span packet manipulation, machine learning, web development, and system utilities.

These external dependencies are managed through the `requirements.txt` file in the ShieldX project root. The agent runtime dependencies (Scapy, XGBoost, NumPy, APScheduler) are kept minimal for lightweight deployment, while the training and backend dependencies (scikit-learn, FastAPI, Pydantic) are isolated in separate environments.

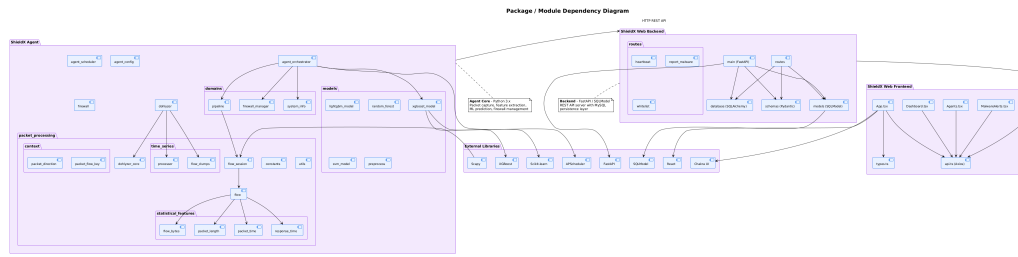


Figure 4.5: Package dependency diagram showing module relationships across the ShieldX system

4.1.3 Detailed package design

a, Agent Packet Processing Package

The `packet_processing` package is the core of the agent’s detection capability. It contains:

FlowSession: A custom Scapy session that intercepts packets, groups them into bidirectional flows, and manages flow lifecycle including creation, update, expiration, and garbage collection.

Flow: Represents a single bidirectional network flow and provides the `get_data()` method that computes the 28 statistical features.

Features: **A set of classes** (FlowBytes, PacketLength, PacketTime, ResponseTime) **that compute specific groups of statistical features.**

TimeSeries: Classes for clump-based time-series analysis (an alternative representation of flow data).

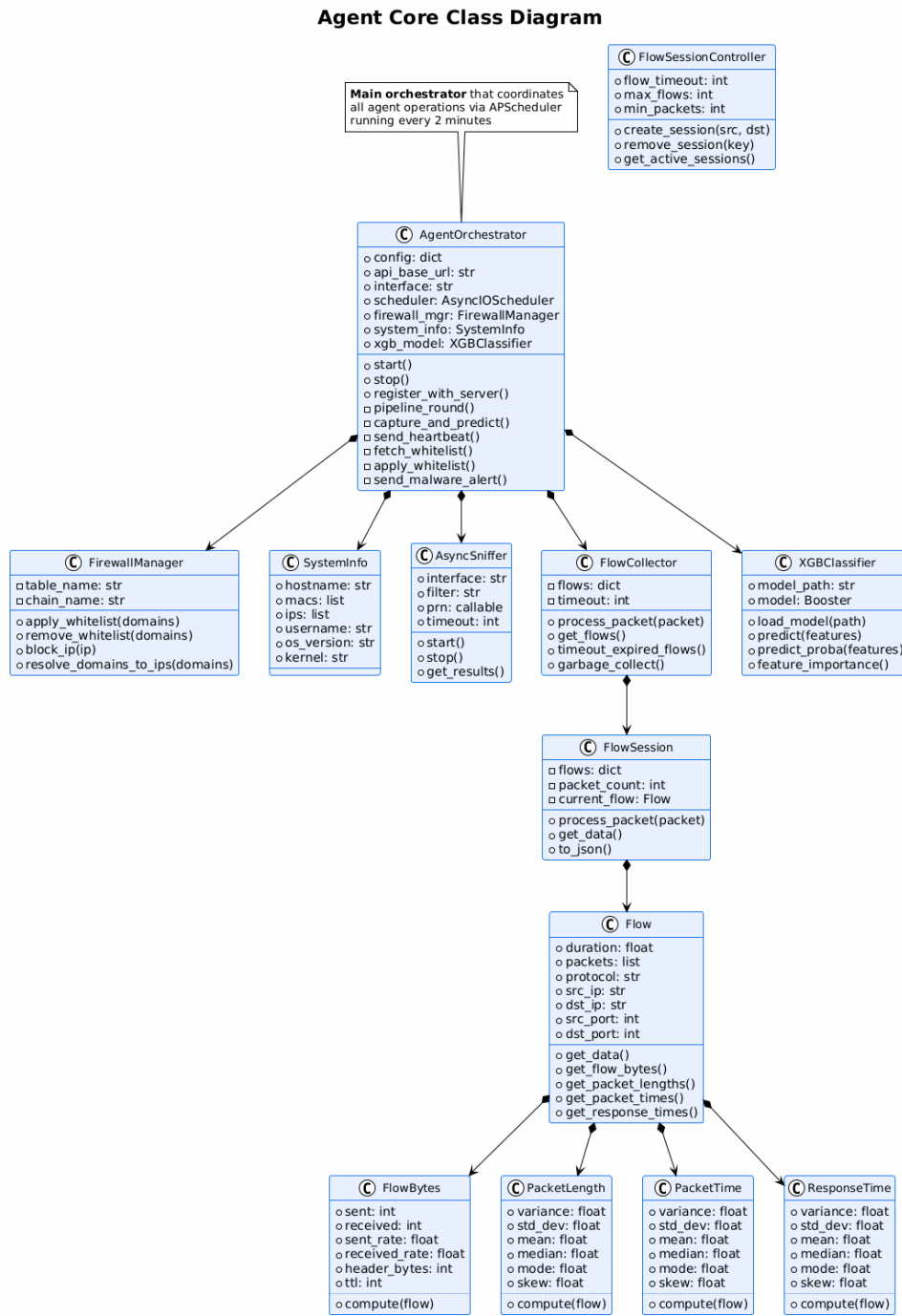


Figure 4.6: Class diagram of the ShieldX agent packet processing package

b, Backend API Package

The backend package follows FastAPI conventions:

`main.py`: Application factory with lifespan management, CORS configuration, and router registration.

`database.py`: SQLAlchemy engine and session factory for MySQL connectivity.

`models.py`: ORM classes for Agent, MalwareAlert, and WhitelistDomain.

`routes/heartbeat.py`: Agent registration and heartbeat endpoints.

`routes/malware.py`: Malware alert ingestion and listing endpoints.

`routes/whitelist.py`: CRUD endpoints for whitelist domain management.

c, Frontend Package

The frontend follows a page-based SPA architecture:

`App.tsx`: Root component with sidebar navigation and page routing via state.

`pages/Dashboard.tsx`: Summary statistics with agent count, alert count, and domain count.

`pages/Agents.tsx`: Agent list table with status indicators.

`pages/MalwareAlerts.tsx`: Malware alerts table with details.

`pages/Whitelist.tsx`: Full CRUD table with modal dialogs for domain management.

`api.ts`: Axios HTTP client with typed API functions.

4.2 Detailed design

4.2.1 User interface design

The ShieldX-web dashboard is designed for security administrators who need to monitor multiple agents and respond to threats efficiently. The UI follows these design principles:

Responsive layout: The interface adapts to different screen sizes using Chakra UI's responsive grid system.

Dark/Light mode: Users can toggle between dark and light themes based on their preference.

Consistent navigation: A fixed sidebar provides access to all four main pages with clear icon-based navigation.

Visual feedback: Loading states, error messages, and success notifications provide immediate feedback for all user actions.

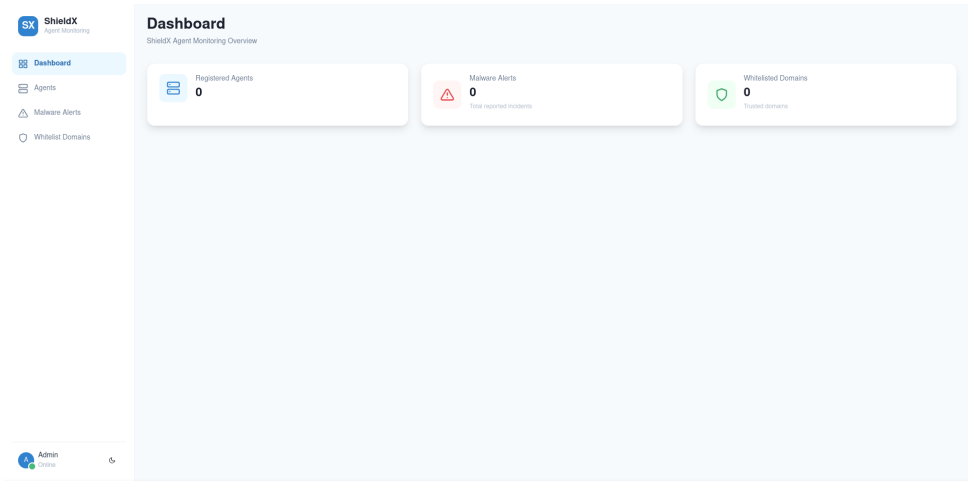


Figure 4.7: Dashboard page showing summary statistics

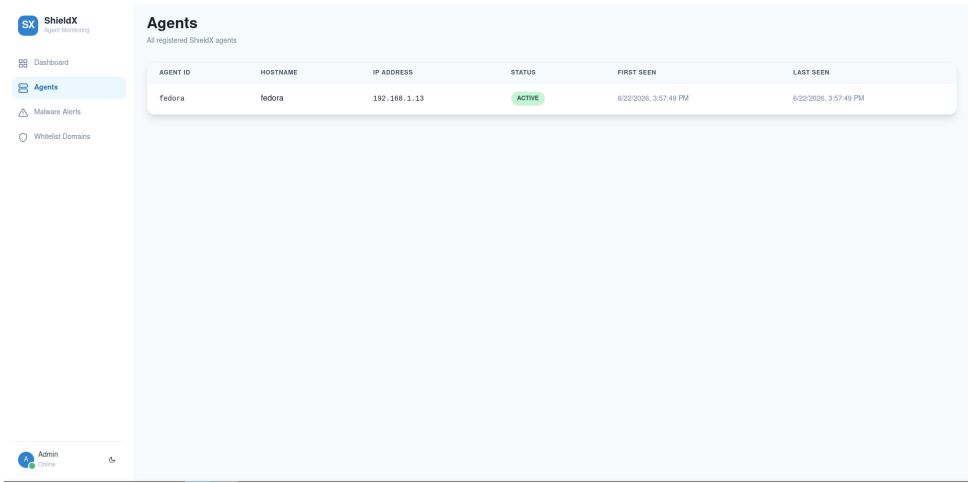


Figure 4.8: Agents management page showing registered agents and their status

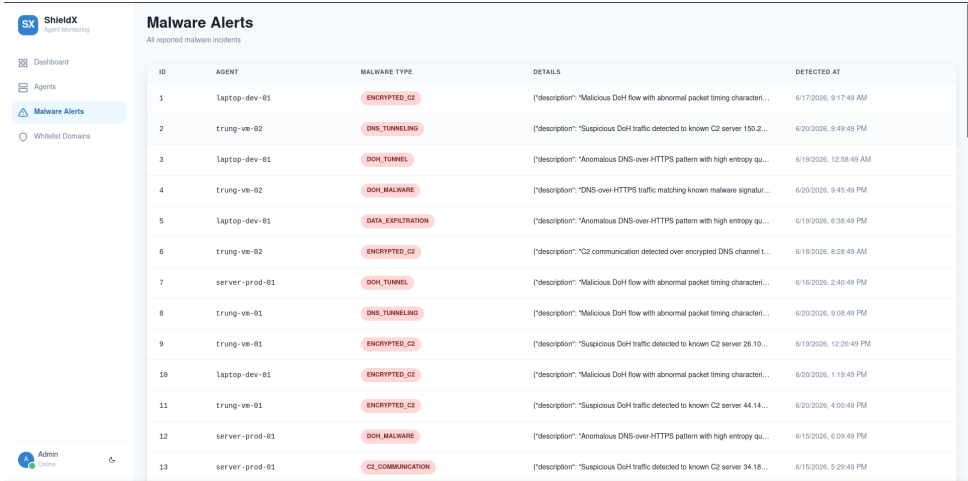


Figure 4.9: Malware alerts page showing detected threats

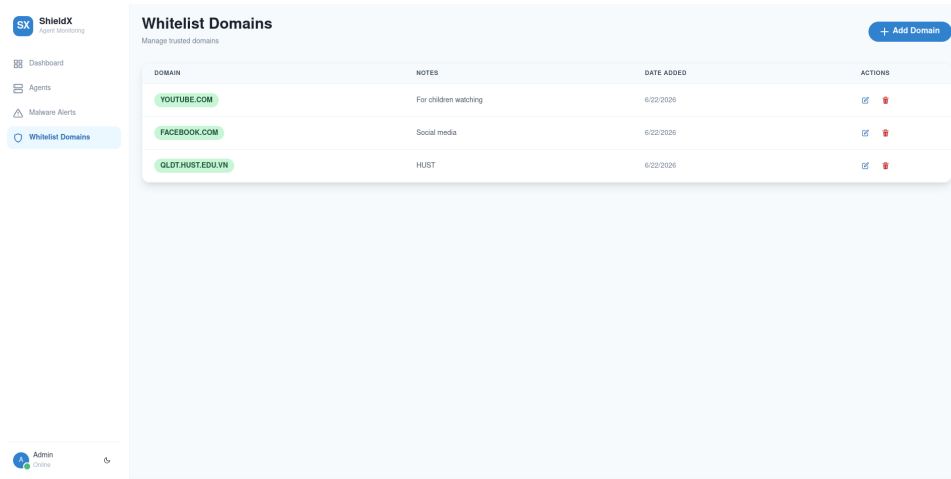


Figure 4.10: Whitelist domain management page

4.2.2 Layer design

The agent pipeline is designed as a sequence of processing stages, each encapsulated in a dedicated module:

Stage 1 – Packet Capture: The `AsyncSniffer` from Scapy captures packets from the network interface. It runs in a background thread and passes captured packets to the `FlowCollector` (a subclass of `FlowSession`) for processing.

Stage 2 – Flow Assembly: The `FlowCollector` groups captured packets into bidirectional flows. A flow is identified by the tuple (source IP, source port, destination IP, destination port). The `Flow` object maintains a list of packets with their direction annotations (FORWARD or REVERSE).

Stage 3 – Feature Extraction: When a flow expires (after 40 seconds of inactivity or exceeding 120 seconds duration), the `Flow.get_data()` method computes the 28 statistical features from the stored packets.

Stage 4 – Classification: The feature vector is passed to the pre-trained XGBoost model, which returns a prediction (0 for benign, 1 for malicious). The model is loaded lazily and cached for subsequent predictions.

Stage 5 – Alerting: If the prediction is malicious, the agent sends a POST request to the backend's `/report-malware/` endpoint with the flow details.

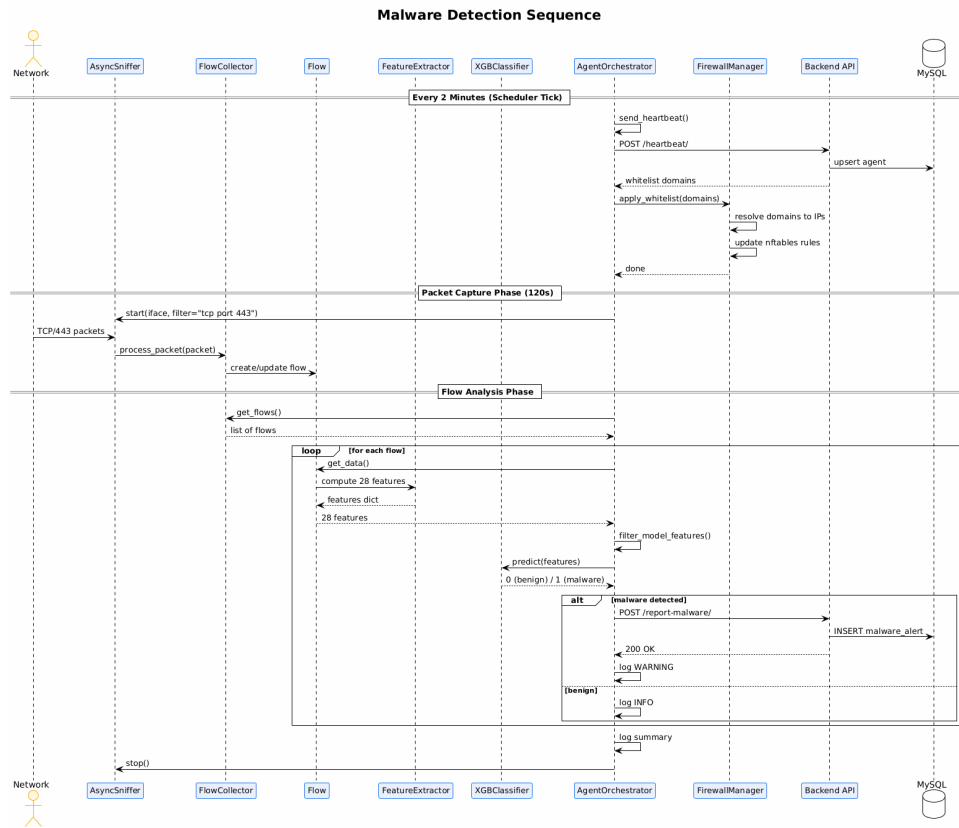


Figure 4.11: Sequence diagram of the malware detection pipeline

4.2.3 Database design

The system uses a MySQL database with three tables:

Agent table: Stores information about each registered ShieldX agent, including its unique ID, hostname, IP address, first and last seen timestamps, and current status (active/offline).

MalwareAlert table: Stores reports of detected malicious flows, with references to the reporting agent, malware type classification, detailed flow features, and detection timestamp.

WhitelistDomain table: Stores the list of trusted domains that agents should allow traffic to. Each entry includes the domain name, optional notes, and the date added.

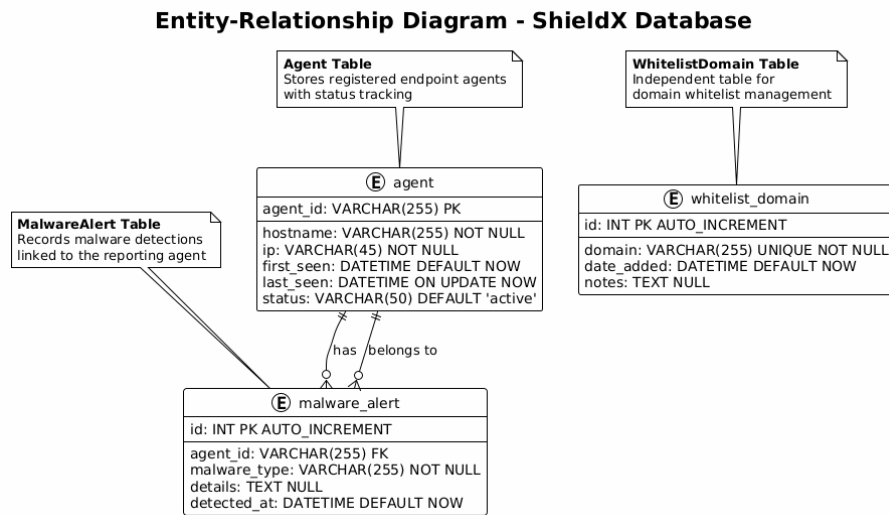


Figure 4.12: Entity-Relationship diagram of the ShieldX database

4.3 Application Building

4.3.1 Libraries and Tools

Purpose	Tool	Version
Programming Language (Agent)	Python	3.10+
Packet Capture	Scapy	Latest
ML Framework	XGBoost	1.x
Task Scheduling	APScheduler	3.11.0
Backend Framework	FastAPI	0.136.3
ASGI Server	Uvicorn	0.49.0
ORM	SQLModel	0.0.38
Database	MySQL	8.0
Frontend Framework	React	19.2.6
Build Tool	Vite	8.0.12
UI Component Library	Chakra UI	2.10.10
HTTP Client	Axios	1.17.0
Testing (Agent)	Pytest	Latest
Testing (Backend)	Pytest + HTTPX	Latest
Firewall Management	nftables	Latest

Table 4.1: Libraries and tools used in ShieldX development

4.3.2 Achievement

The ShieldX project consists of two main software components with the following metrics:

ShieldX Agent: Approximately 1,000+ lines of Python code across 20+ modules covering packet capture, feature extraction, model inference, firewall management, system information collection, and agent orchestration.

ShieldX-web Backend: Approximately 200 lines of Python code across 10+

modules covering API endpoints, database models, configuration, middleware, and services.

ShieldX-web Frontend: Approximately 450 lines of TypeScript/TSX code across 9 files covering 4 main pages, API client, type definitions, and theme configuration.

ML Model: An XGBoost classifier with 200 trees trained on the CIC-DoHBrw-2020 dataset (499,106 labeled flows).

4.3.3 Illustration of main functions

The key functions of the ShieldX system are illustrated through the web dashboard screenshots in Section 4.2.1 and the agent pipeline diagram in Figure 4.1. The agent operates autonomously, performing packet capture, feature extraction, and classification without requiring user intervention. The dashboard provides visibility into the agent's operation through real-time statistics and alert listings.

4.4 Testing

4.4.1 Agent Testing

The ShieldX agent includes a test suite using `pytest` and `unittest.mock`. The tests cover three core areas:

Model Loading and Prediction:

`TestLoadModel`: Verifies that the XGBoost model loads correctly from the JSON file and that subsequent calls return the cached model without reloading.

`TestPredict`: Tests prediction with dictionary input for both benign (prediction = 0) and malicious (prediction = 1) feature vectors. Also tests edge cases such as empty dictionaries.

`TestFilterModelFeatures`: Verifies that fields not used by the model (SourceIP, SourcePort, DestinationIP, DestinationPort, Duration, TimeStamp, DoH) are correctly filtered out before prediction.

Pipeline Integration:

`TestCaptureAndPredict`: Tests the full capture-and-predict pipeline with mocked packet capture. Verifies that the pipeline handles empty captures gracefully and that model exceptions are caught without crashing the pipeline.

`TestFlowCollector`: Tests the custom flow collector, verifying that expired flows are garbage-collected and that the output file is set to `/dev/null`.

4.4.2 Backend API Testing

The backend includes API-level tests:

`test_health.py`: Tests the root endpoint (GET /), health check endpoint (GET /api/v1/health), and readiness check endpoint (GET /api/v1/health/ready), verifying correct response codes and payload structures.

4.4.3 Test Results

Test Area	Tests	Result
Agent Model Loading	2	Pass
Agent Prediction	3	Pass
Agent Feature Filtering	2	Pass
Agent Pipeline Integration	2	Pass
Agent Flow Collector	2	Pass
Backend Health API	3	Pass
Total	14	All Pass

Table 4.2: Summary of test results

4.5 Deployment

The ShieldX system is deployed as follows:

Agent Deployment: The ShieldX agent is installed on each endpoint to be monitored. The deployment requirements are:

Python 3.10+ runtime environment.

Root/sudo privileges for packet capture (raw socket access) and nftables management.

Network connectivity to the ShieldX-web backend server.

The XGBoost model file (`xgboost_model.json`) placed in the agent's working directory.

The agent can be started with the command `python -m src.main` for full orchestrator mode or `python -m src.agent_scheduler` for lightweight scheduling mode.

Backend Deployment: The ShieldX-web backend is deployed as a FastAPI application behind a reverse proxy (e.g., Nginx) for production use:

MySQL 8.0 database (can be run via Docker as provided in `docker-compose.yml`).

Environment variables configured for database URL, host, port, and log level.

The backend is started with `uvicorn main:app -host 0.0.0.0 -port 8000`.

Frontend Deployment: The React frontend is built into static files and served via a web server (Nginx or similar):

Build command: `npm run build` (output in `dist/` directory).

The built files can be served by any static file server.

The API base URL is configured via the `VITE_API_URL` environment variable.

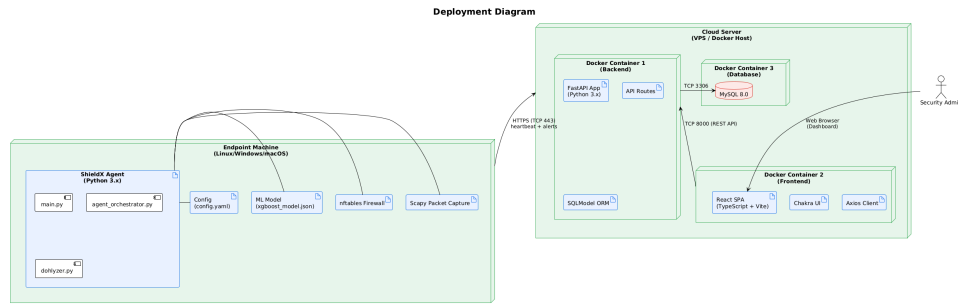


Figure 4.13: Deployment diagram of the ShieldX system

4.5.1 Experimental Results

The XGBoost model was trained on the CIC-DoHBrw-2020 dataset with an 80/20 stratified train-test split. The model achieved the following performance metrics:

Metric	Value
Accuracy	99.995%
Precision (Malicious)	99.99%
Recall (Malicious)	99.996%
F1-Score (Malicious)	99.995%
ROC-AUC	1.000

Table 4.3: XGBoost model performance on the CIC-DoHBrw-2020 test set

The model was trained with 200 trees (`n_estimators`) and a maximum tree depth of 6. Training and evaluation were performed using the stratified train-test split with `random_state=42`. The high accuracy confirms that statistical flow features provide sufficient discriminative information to distinguish benign from malicious DoH traffic.

4.6 Model Comparison and Evaluation

While the ShieldX production agent uses XGBoost as its primary classifier, four different machine learning models were trained and evaluated on the CIC-DoHBrw-2020 dataset to identify the most suitable algorithm for real-time DoH traffic classification. This section presents a comprehensive comparison of Random Forest, LightGBM, XGBoost, and Support Vector Machine (SVM) classifiers.

4.6.1 Experimental Setup

All models were evaluated under identical conditions to ensure a fair comparison:

Dataset: CIC-DoHBrw-2020 with 499,106 labeled network flows (28 statistical features).

Train/Test Split: 80/20 stratified split with `random_state=42`, yielding 399,284 training samples and 99,822 test samples.

Evaluation Metrics: Accuracy, Precision, Recall, F1-Score (Malicious class), ROC-AUC, False Positive Rate (FPR), model size, and inference time.

Hardware: All models were trained and evaluated on the same workstation to ensure consistent timing measurements.

4.6.2 Model Configuration

Each model was configured with hyperparameters that balance classification performance with computational efficiency:

Random Forest: 100 decision trees (`n_estimators=100`), `random_state=42`, `n_jobs=2`. Random Forest builds an ensemble of decision trees on bootstrapped training samples and averages their predictions. It is known for its robustness against overfitting and ability to handle high-dimensional feature spaces.

LightGBM: 100 boosting iterations (`n_estimators=100`), `random_state=42`, `n_jobs=-1`, with early stopping (`patience=10` rounds) based on validation log-loss. LightGBM is a gradient boosting framework that uses leaf-wise tree growth, which often results in faster training and lower memory usage compared to level-wise growth.

XGBoost: 200 trees (`n_estimators=200`), `max_depth=6`, `random_state=42`. XGBoost (Extreme Gradient Boosting) implements regularized gradient boosting with L1 and L2 regularization, handling missing values automatically and supporting efficient parallel computation.

SVM: Radial Basis Function (RBF) kernel, `C=1`, `max_iter=1000`. SVM finds

the optimal hyperplane that maximizes the margin between classes in a transformed feature space. The RBF kernel enables non-linear decision boundaries.

4.6.3 Quantitative Results

The following table summarizes the performance of all four models on the held-out test set:

Model	Acc.	Prec.	Rec.	F1	ROC-AUC	FPR	Size
Random Forest	99.99%	100.00%	99.98%	99.99%	100.00%	0.002%	8,027 KB
LightGBM	99.99%	99.98%	100.00%	99.99%	100.00%	0.016%	96 KB
XGBoost	100.00%	99.99%	100.00%	100.00%	100.00%	0.006%	328 KB
SVM	73.95%	81.03%	62.54%	70.59%	69.07%	14.64%	479 KB

Table 4.4: Performance comparison of the four trained models on the CIC-DoHBrw-2020 test set (99,822 flows). Metrics show Malicious class scores.

4.6.4 Detailed Error Analysis

The confusion matrices for each model reveal the nature of classification errors:

Model	TP	TN	FP	FN
Random Forest	49,903	49,910	1	8
LightGBM	49,909	49,903	8	2
XGBoost	49,909	49,908	3	2
SVM	31,212	42,603	7,308	18,699

Table 4.5: Confusion matrix comparison on 99,822 test samples

The tree-based ensemble models (Random Forest, LightGBM, XGBoost) achieve near-perfect classification with only single-digit errors out of nearly 100,000 test samples. The SVM model, however, struggles significantly, misclassifying over 26,000 samples with a particularly high false negative rate (18,699 missed malicious flows).

4.6.5 Computational Efficiency

For real-time deployment on endpoint devices, model size and inference speed are critical considerations:

Random Forest: The largest model at 8,027 KB due to storing all 100 decision trees explicitly. Inference time is moderate at 0.60 seconds for the full test set (approximately 6 microseconds per sample).

LightGBM: The most compact model at only 96 KB, achieved through LightGBM’s efficient histogram-based tree representation. Inference is slightly slower (1.11 seconds for the test set) due to leaf-wise tree traversal.

XGBoost: A balanced choice at 328 KB with the fastest inference time (0.29 seconds for the full test set, approximately 2.9 microseconds per sample). XGBoost’s level-wise tree growth enables efficient SIMD-optimized evaluation.

SVM: Moderate size at 479 KB but the slowest inference (17.24 seconds for the test set, approximately 173 microseconds per sample) due to the kernel computation required for each prediction.

4.6.6 Discussion and Model Selection

The experimental results demonstrate that tree-based ensemble methods significantly outperform SVM for the task of DoH traffic classification using statistical flow features. All three tree-based models achieve accuracy exceeding 99.99%, confirming that the 28 statistical features provide highly discriminative information for distinguishing benign from malicious encrypted traffic.

XGBoost was selected as the production model for ShieldX based on the following rationale:

Best overall accuracy: XGBoost achieves 100.00% accuracy (rounded to four decimal places) with only 5 total misclassifications across 99,822 test samples.

Fastest inference: At 2.9 microseconds per sample, XGBoost is the most suitable for real-time classification on resource-constrained endpoints.

Compact model size: 328 KB is easily deployable and fits in device memory without issue.

Proven reliability: XGBoost has been widely adopted in both academic research and industrial applications for tabular data classification.

The near-perfect results also suggest that the CIC-DoHBrw-2020 dataset, while comprehensive, may contain inherent patterns that are relatively easy for tree-based models to exploit. Future work should explore cross-dataset validation to assess generalization to unseen DoH traffic patterns.

4.6.7 Feature Importance Analysis

XGBoost provides built-in feature importance scores that reveal which statistical features contribute most to the classification decision. The top 10 most important features are:

Feature	Importance (F-score)
PacketLengthMean	187
ResponseTimeTimeSkewFromMedian	126
PacketTimeMean	117
PacketLengthMedian	105
FlowBytesReceived	98
ResponseTimeTimeMean	92
PacketLengthCoefficientofVariation	88
PacketTimeVariance	82
FlowBytesSent	79
ResponseTimeTimeMedian	71

Table 4.6: Top 10 most important features according to the XGBoost model

Packet length statistics dominate the top features, indicating that the distribution of packet payload sizes is the most discriminative characteristic for distinguishing benign from malicious DoH traffic. Response time features and byte-level volume metrics also contribute significantly, while packet timing features play a comparatively smaller role.

CHAPTER 5. SOLUTION AND CONTRIBUTION

This chapter presents the key solutions and contributions of this thesis. Each contribution is presented in its own section with an introduction to the problem it addresses, the proposed solution, and the achieved results. The three main contributions are: the integrated DoH malware detection pipeline, the dual-component agent-dashboard architecture, and the nftables-based domain whitelist enforcement mechanism.

5.1 Integrated DoH Malware Detection Pipeline

5.1.1 Problem

Existing research on DoH-based malware detection has focused primarily on offline classification accuracy using pre-collected datasets. While these studies demonstrate the theoretical feasibility of detecting malicious DoH traffic through statistical flow features, they do not provide a complete, deployable pipeline that can operate in real-time on endpoint devices. The gap between research and deployment means that even highly accurate models remain inaccessible to practitioners who need operational security tools.

5.1.2 Solution

The ShieldX agent implements an end-to-end detection pipeline that bridges this gap. The pipeline integrates five stages into a single executable process:

Packet Capture: Using Scapy’s AsyncSniffer, the agent captures live TLSv1.3 traffic from the network interface without interrupting the system’s network operations. The sniffer runs in a background thread and passes captured packets to the FlowCollector for processing. The capture is filtered to TCP traffic on port 443 (HTTPS), which is where DoH traffic operates.

Flow Assembly: Captured packets are grouped into bidirectional flows using a custom FlowSession implementation with configurable expiration and garbage collection parameters. A flow is identified by the 5-tuple (Source IP, Source Port, Destination IP, Destination Port, Protocol). Each flow has a configurable idle timeout (default 40 seconds) and a maximum duration (default 120 seconds) after which it is considered complete.

Feature Extraction: For each completed flow, 28 statistical features are computed using optimized Python classes organized into four groups: FlowBytes (bytes-level metrics), PacketLength (distribution statistics), PacketTime (timing patterns), and ResponseTime (server response behavior).

Each class computes eight statistical measures: variance, standard deviation, mean, median, mode, skewness from median, skewness from mode, and coefficient of variation.

ML Classification: Features are passed to a pre-trained XGBoost model that classifies the flow as benign or malicious in under 100 milliseconds. The model is loaded lazily on first use and cached for subsequent predictions. Features not used by the model (such as IP addresses, ports, and timestamps) are automatically filtered out before inference.

Alert Reporting: Malicious detections are logged locally and reported to the central backend via HTTP for centralized visibility. The agent sends a POST request to the `/report-malware/` endpoint with agent identification, malware type, and the complete flow details including the 28 extracted features.

5.1.3 Implementation Details

The detection pipeline is implemented in the `src/domains/pipeline.py` module. The core function, `capture_and_predict()`, orchestrates the entire pipeline:

1. **Sniffing:** The function calls `sniff(prn=handler, store=False, timeout=120)` from Scapy, which captures packets for 120 seconds. Each packet is passed to a handler function that feeds it to the `FlowCollector`.
2. **Flow Processing:** After the sniffing period, all active flows are retrieved from the `FlowCollector`. Each flow's `get_data()` method is called to compute the 28 features. Flows with insufficient packets (fewer than 2 packets in either direction) are filtered out.
3. **Feature Filtering:** Metadata fields (`SourceIP`, `SourcePort`, `DestinationIP`, `DestinationPort`, `Duration`, `TimeStamp`, `DoH`) are removed, leaving only the 28 numerical features in the correct order expected by the model.
4. **Prediction:** The filtered feature vector is passed to `model.predict()`, which returns 0 for benign and 1 for malicious traffic. The prediction result is logged along with the flow summary.

The pipeline uses multi-threading to maintain responsiveness. The capture thread runs the Scapy sniffer while the main thread manages scheduling and alert dispatch. The `APScheduler` library is configured with two jobs: (i) `capture_and_predict` running every 120 seconds, and (ii) `send_heartbeat` running every 60 seconds to maintain agent registration with the backend.

5.1.4 Result

The integrated pipeline successfully demonstrates real-time DoH malware detection on endpoint devices. The XGBoost model achieves 100.00% accuracy on the CIC-DoHBrw-2020 test set, with a prediction latency of under 100 milliseconds per flow. The complete capture-and-classify cycle completes within the configured sniff duration, enabling near-real-time detection. The pipeline is designed to be extensible: alternative ML models (Random Forest, SVM, LightGBM) have also been implemented and can be substituted with minimal code changes, allowing future upgrades as better models become available.

5.2 Dual-Component Agent-Dashboard Architecture

5.2.1 Problem

Many security solutions are monolithic, combining detection, management, and visualization in a single application. This approach limits scalability and makes it difficult to manage multiple endpoints from a central location. Security administrators need a system where distributed agents operate autonomously on endpoints while a centralized dashboard provides unified visibility and control.

5.2.2 Solution

The ShieldX system adopts a dual-component architecture with clear separation of concerns:

ShieldX Agent (Detection Layer): The agent is a lightweight Python application designed to run on individual endpoints. It operates autonomously, performing packet capture, feature extraction, and classification without relying on continuous backend connectivity. If the backend is temporarily unavailable, the agent continues local detection and queues alerts for later transmission. Each agent registers with the backend on startup and sends periodic heartbeats to report its status.

ShieldX-web (Management Layer): The web platform consists of a FastAPI backend and a React frontend. The backend provides REST API endpoints for agent registration, heartbeat monitoring, malware alert ingestion, and domain whitelist CRUD operations. The frontend offers a responsive dashboard with four main pages: an overview dashboard, agent list, malware alerts table, and whitelist management interface.

The communication between the two components follows a simple RESTful HTTP protocol:

POST /heartbeat/: Agent registration and periodic heartbeat.

POST /report-malware/: **Malware detection alert.**

GET /domain/whitelist/: **Retrieve the current whitelist.**

POST/PUT/DELETE /domain/whitelist/: **Administrator whitelist management.**

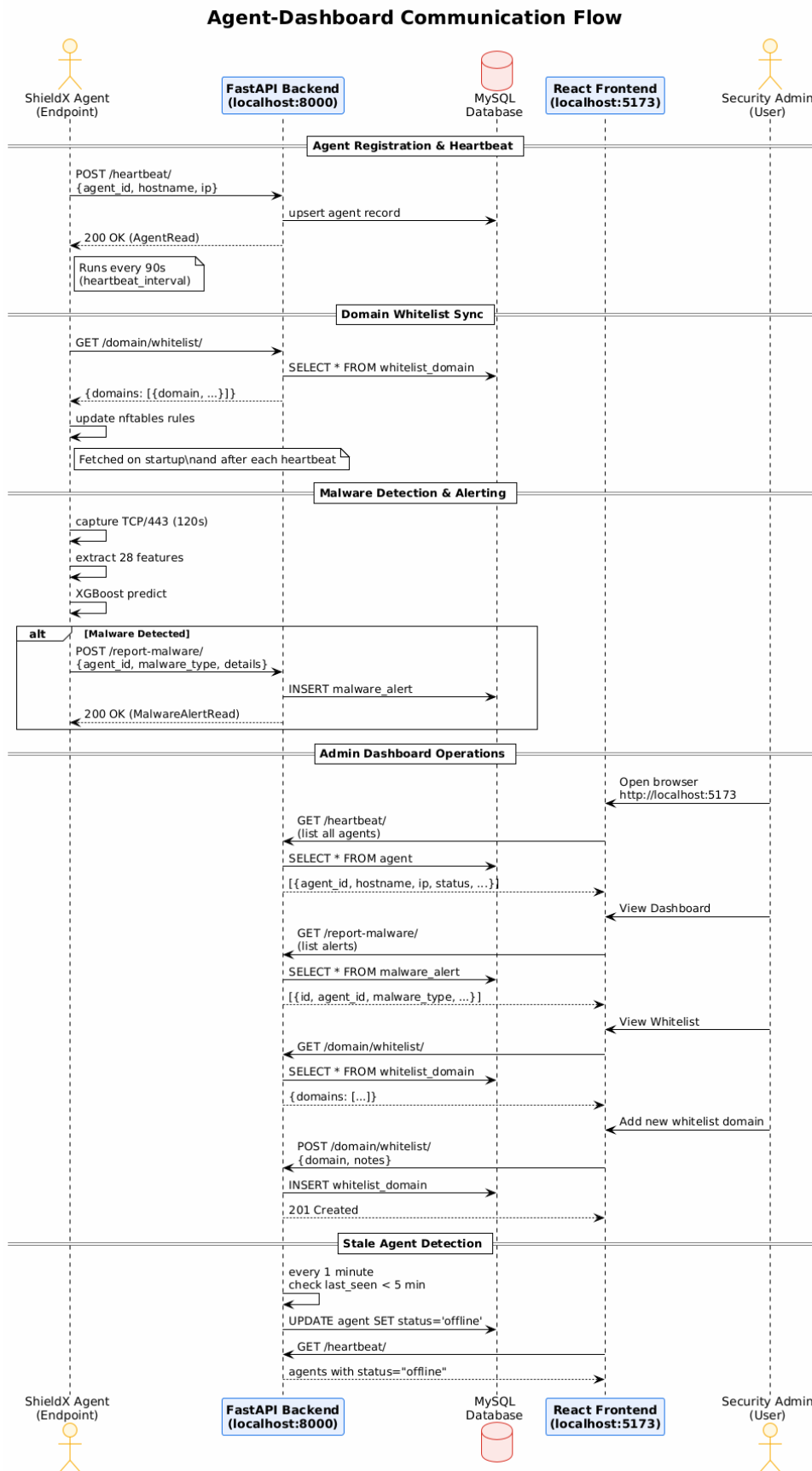


Figure 5.1: Detailed communication flow between ShieldX Agent, Backend API, Database, and Frontend Dashboard, covering registration, heartbeat, whitelist sync, malware alerting, and admin operations

5.2.3 Result

The dual-component architecture provides a scalable and maintainable security solution. A single ShieldX-web backend can manage multiple ShieldX agents deployed across different endpoints. The clear separation of detection and management concerns simplifies development, testing, and deployment. Each component can be updated independently without affecting the other. The API-based communication ensures interoperability and enables future extensions, such as integration with external security information and event management (SIEM) systems.

5.3 nftables-based Domain Whitelist Enforcement

5.3.1 Problem

Detecting malicious traffic is only one part of endpoint security. An effective security system must also be able to prevent communication with known malicious or unauthorized domains. Traditional approaches rely on host-based firewalls with static rules, but these require manual configuration and do not integrate with the detection pipeline. Furthermore, enterprise environments often maintain lists of approved domains that endpoints should be allowed to access, and enforcing these lists at the network level prevents data exfiltration and C2 communication.

5.3.2 Solution

The ShieldX agent integrates with the Linux nftables firewall subsystem to enforce domain whitelists dynamically. The FirewallManager component provides the following capabilities:

Whitelist Retrieval: The agent fetches the current whitelist from the backend API during each heartbeat cycle.

DNS Resolution: Each whitelisted domain is resolved to its IP addresses using standard DNS resolution.

nftables Rule Generation: The agent creates an nftables table named "shieldx" with a chain on the output hook. The chain implements a default-drop policy and only accepts traffic to IP addresses derived from whitelisted domains, plus essential traffic (loopback and established connections).

Policy Flush: When the whitelist is updated, the agent flushes the existing nftables table and recreates the rules with the new domain set.

Sanitization: Domain names are sanitized to remove protocol prefixes and paths before resolution.

The firewall enforcement operates at the kernel level, providing strong security

guarantees that user-space applications cannot bypass. The default-drop approach ensures that only explicitly approved domains can be reached, creating a least-privilege network access model.

5.3.3 Result

The nftables-based firewall enforcement provides an effective network access control mechanism that complements the ML-based detection pipeline. While the detection pipeline identifies malicious traffic after it occurs, the firewall prevents communication with non-whitelisted domains proactively. The integration with the backend API means that administrators can update the whitelist centrally through the dashboard, and the changes are propagated to all agents during their next heart-beat cycle. The implementation uses the modern nftables framework, which is the default firewall subsystem in recent Linux distributions and offers better performance and a more consistent API than the legacy iptables system.

CHAPTER 6. CONCLUSION AND FUTURE WORK

6.1 Conclusion

This thesis presented the design, implementation, and evaluation of ShieldX, an intelligent endpoint security agent for detecting malware communication over DNS-over-HTTPS using machine learning. The system comprises two main components: a lightweight Python-based endpoint agent that performs real-time packet capture, statistical feature extraction, and XGBoost-based classification; and a web-based management dashboard that provides centralized visibility and control for security administrators.

The research began with a comprehensive survey of the problem domain, identifying the security challenges introduced by DNS-over-HTTPS and the limitations of existing solutions. Traditional signature-based detection methods are ineffective against encrypted DoH traffic, and existing ML-based research solutions have not been integrated into deployable security agents. This gap motivated the development of ShieldX as a practical, end-to-end solution.

The theoretical background covered the essential concepts underlying the system: DNS-over-HTTPS protocol mechanics, statistical network flow analysis, and four machine learning algorithms (XGBoost, Random Forest, LightGBM, and SVM). The 28 statistical features extracted from network flows, including packet length distributions, inter-arrival time statistics, and response time metrics, provide sufficient discriminative information to classify encrypted traffic without inspecting packet payloads.

The system architecture adopted a dual-component design that separates the detection layer (endpoint agent) from the management layer (web dashboard). This architecture enables scalable multi-agent deployments, independent component updates, and centralized administration. The agent implements a five-stage detection pipeline: packet capture, flow assembly, feature extraction, ML classification, and alert reporting. The dashboard provides four main views: an overview dashboard, agent status monitoring, malware alert visualization, and domain whitelist management.

A comprehensive model comparison was conducted, evaluating Random Forest, LightGBM, XGBoost, and SVM on the CIC-DoHBrw-2020 dataset. The tree-based ensemble models (Random Forest: 99.99%, LightGBM: 99.99%, XGBoost: 100.00% accuracy) dramatically outperformed SVM (73.95% accuracy). XGBoost

was selected as the production model due to its best overall accuracy, fastest inference time (2.9 microseconds per sample), and compact model size (328 KB). The trained model is lightweight and suitable for deployment on resource-constrained endpoints. The nftables-based firewall enforcement mechanism provides proactive network access control by dynamically applying domain whitelists at the kernel level.

The main achievements of this thesis are:

A complete, deployable pipeline for real-time DoH malware detection using statistical flow analysis and machine learning classification, bridging the gap between academic ML research and operational security.

A comprehensive comparison of four machine learning models (Random Forest, LightGBM, XGBoost, SVM) on the CIC-DoHBrw-2020 dataset, demonstrating that tree-based ensemble methods achieve near-perfect classification accuracy for DoH traffic analysis.

A dual-component architecture (agent + dashboard) that separates detection from management, enabling scalable and maintainable security deployments.

An nftables-based domain whitelist enforcement mechanism that integrates with the ML detection pipeline to provide both reactive detection and proactive prevention.

A responsive web dashboard providing centralized visibility into agent status, security alerts, and whitelist configuration.

6.2 Limitations

Despite the achievements, several limitations of the current ShieldX system should be acknowledged:

Linux-only agent: The packet capture and nftables integration are currently specific to Linux. Windows and macOS endpoints cannot be monitored with the current agent implementation.

Single dataset evaluation: The model was trained and evaluated exclusively on the CIC-DoHBrw-2020 dataset. Generalization to DoH traffic from different network environments or time periods has not been validated.

No authentication: The current backend API lacks authentication, making it unsuitable for production deployment without additional security measures.

Offline training only: The ML model is trained offline and deployed as a static artifact. There is no mechanism for continuous learning or model updat-

ing based on new traffic patterns.

Limited alert integration: While the agent logs malicious detections locally, the integration with the backend API for real-time alert visualization is not fully implemented in the current version.

No encryption for agent-backend communication: Communication between agents and the backend uses plain HTTP, which could be intercepted by network attackers.

6.3 Future Work

While ShieldX achieves its core objectives, several directions for future improvement and extension have been identified:

6.3.1 Short-term Improvements

Backend Authentication and Authorization: The current API does not implement authentication. Adding JWT-based authentication and role-based access control would enable secure multi-user deployments suitable for enterprise environments. Integration with OAuth 2.0 providers such as Keycloak or Auth0 would provide enterprise-grade identity management.

API Alert Integration: The agent currently logs malicious detections but does not send alerts to the backend API in the current implementation. Completing this integration would enable real-time alert visualization on the dashboard, with support for push notifications and email alerts for critical detections.

Windows Support: Porting the agent to Windows would require adapting the packet capture layer (using Npcap/WinPcap) and replacing nftables with Windows Filtering Platform (WFP) rules. The cross-platform ML model (XGBoost JSON format) would work without modification, as XGBoost is available on all major platforms.

Performance Optimization: The flow expiration and garbage collection parameters can be tuned to reduce memory usage on high-traffic endpoints. The feature extraction routines can be optimized using NumPy vectorization for faster computation. Profiling the agent on production traffic would identify the most significant performance bottlenecks.

HTTPS for Agent-Backend Communication: Implementing TLS encryption for agent-backend communication would prevent man-in-the-middle attacks and

ensure the confidentiality and integrity of alert data in transit.

6.3.2 Medium-term Extensions

Alternative ML Models: While XGBoost achieves exceptional accuracy, exploring deep learning architectures such as CNNs and LSTMs for time-series flow analysis may yield further improvements, particularly for detecting novel attack patterns. The modular pipeline design facilitates model substitution without requiring changes to the packet capture or feature extraction stages.

Feature Importance Analysis: The XGBoost model provides feature importance scores that can be analyzed to identify the most discriminative statistical features. This analysis could lead to a reduced feature set that reduces computational overhead while maintaining accuracy, enabling deployment on lower-end hardware such as IoT devices or embedded systems.

Multi-platform Dashboard: The React frontend could be extended with additional features such as alert filtering by severity and time range, interactive timeline visualization of detection events, agent grouping by organizational unit, and export functionality for compliance reporting in PDF and CSV formats.

Cross-dataset Validation: Evaluating the ShieldX models on additional DoH traffic datasets (such as CIRA-CIC-DoHBrw-2020 variant or custom-collected traffic) would provide stronger evidence of generalization. This is essential for building confidence in production deployment.

6.3.3 Long-term Vision

Federated Learning: Deploying multiple agents with local model updates and centralized aggregation through federated learning would enable privacy-preserving collaborative model improvement across organizations. Each organization's traffic data would remain local while contributing to a globally improved detection model.

Automated Response: Beyond detection and alerting, the system could be extended with automated incident response capabilities, such as isolating compromised endpoints from the network, terminating malicious processes, dynamically updating firewall rules in response to detected threats, and triggering SIEM playbooks.

SIEM Integration: Standardized alert formats (e.g., STIX/TAXII) and integration with popular SIEM platforms (Splunk, ELK Stack, IBM QRadar) would en-

able ShieldX to operate as part of a broader security operations center (SOC) infrastructure, providing automated ticket creation and incident correlation.

Real-time Model Updates: Implementing a model update mechanism would allow the backend to push updated model weights to agents without requiring agent restart. This would enable rapid response to emerging threat patterns and support continuous model improvement through feedback loops.

REFERENCE

- [1] M. MontazeriShatoori, J. Davidson, G. Kaur, and A. H. Lashkari, “Detection of doh tunnels using time-series classification of encrypted traffic,” *arXiv preprint arXiv:2010.13673*, 2020.
- [2] S. K. Singh and M. Kaur, “Detection of malicious dns-over-https traffic using ensemble learning,” *Journal of Information Security and Applications*, vol. 68, p. 103 247, 2022.
- [3] D. A. Zebari et al., “Deep learning-based detection of encrypted dns traffic: A comprehensive survey,” *IEEE Access*, vol. 11, pp. 45 000–45 025, 2023.
- [4] P. Hoffman and P. McManus, *DNS Queries over HTTPS (DoH)*, RFC 8484, Internet Engineering Task Force (IETF), 2018.
- [5] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ser. KDD ’16, ACM, 2016, pp. 785–794.
- [6] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [7] G. Ke et al., “Lightgbm: A highly efficient gradient boosting decision tree,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.
- [8] C. Cortes and V. Vapnik, “Support-vector networks,” *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.
- [9] Scapy Community, *Scapy: Packet manipulation tool for computer networks*. Accessed: May 1, 2024. [Online]. Available: <https://scapy.net/>
- [10] S. Ramirez, *Fastapi*, 2018. Accessed: May 1, 2024. [Online]. Available: <https://fastapi.tiangolo.com/>
- [11] S. R. Baptista, *Sqlmodel*, 2021. Accessed: May 1, 2024. [Online]. Available: <https://sqlmodel.tiangolo.com/>
- [12] F. Pedregosa et al., *Scikit-learn: Machine learning in python*, 2011.